FPL

Read in the order of the three scopes

FPL

Table of contents

FPL 7
- Goal . . . 7
- What FPL is . . . 7
- Get started . . . 7
  - Try it . . . 7
  - In a container . . . 7
  - Hack on it . . . 8
- Where things are . . . 8
- References . . . 8
- Try it in Marimo . . . 8

1 Bootstrap 12

01.01.01 Source 13

01.01.01 Source 14

01.01.02 FON 15

01.01.02 FON 16

01.01.03 Pipe 30

01.01.03 Pipe 31

01.01.04 Render 36

01.01.04 Render 37

01.02.01 Lex 38

01.02.01 Lex 39

01.02.02 Parse 48

01.02.02 Parse 49

01.02.03 Resolve 61
01.02.03 Resolve 62
01.02.04 Expand 63
01.02.04 Expand 64
01.02.05 Desugar 65
01.02.05 Desugar 66
01.02.06 Elaborate 107
01.02.06 Elaborate 108
01.02.07 Lower 125
01.02.07 Lower 126
01.02.08 Normalize 127
01.02.08 Normalize 128
01.02.09 Optimize 129
01.02.09 Optimize 130
01.02.10 Evaluate 131
01.02.10 Evaluate 132
01.02.11 Codegen 153
01.02.11 Codegen 154
01.03.01 Source 155
01.03.01 Source 156
01.03.02 Tokens 157
01.03.02 Tokens 158
01.03.03 Syntax 159
01.03.03 Syntax 160

01.03.04 Resolved 161

01.03.04 Resolved 162

01.03.05 Expanded 163

01.03.05 Expanded 164

01.03.06 Elaborated 165

01.03.06 Elaborated 166

01.03.07 Core 167

01.03.07 Core 168

01.04.01 Pass 169

01.04.01 Pass 170

01.04.02 Run 171

01.04.02 Run 172

01.04.03 Inspect 175

01.04.03 Inspect 176

01.04.04 Build 177

01.04.04 Build 178

01.05.01 Pipeline 179

01.05.01 Pipeline 180

01.06.01 Advance 181

01.06.01 Advance 182

2 Docs 183

DESIGN.md — the FPL vision 184

1. One sentence 184
2. Why it exists 184
3. Feel 184

4. Non-goals . . . 184
5. Influences . . . 185
6. Core shape (feeds the stable core AST) . . . 185
7. Open questions . . . 185
Which accelerator, and can they be one mechanism . . . 185
Which extension mechanism . . . 186

SPEC.md — the FPL language specification **187**
1. Lexical structure . . . 187
2. Grammar . . . 187
3. Core language . . . 187
4. Evaluation . . . 187
5. Type system . . . 187
6. Errors . . . 188

02.03 ROADMAP.md — milestones **189**
Before milestone 0 . . . 189

02.04.01 2026-09-19 — issue, task, patch **191**
Three objects, and the work each one does . . . 191
Why there is no plan object . . . 192
Two kinds of edge . . . 192
Depth and width, not a calendar . . . 192
Open . . . 193

02.04.02 2026-09-19 — plumbing, not prophecy **194**
What exists on this date . . . 194
For agents . . . 194
Reference . . . 194

02.04.03 2026-09-20 — the project that mediates the language **196**
The pattern . . . 196
What the mediator actually does . . . 196
Why that matters here in particular . . . 197
And if there is no outside project at all . . . 197
Method, and one principle with a precedent . . . 197
The tension this leaves . . . 198
What this does not license . . . 198

3.1 Features **199**

draft1 — spec **200**

draft2 — spec 201

draft3 — spec 202

match — spec 203

server — spec 204

sketch — spec 205

FPL

Dit is de homepage van de repo. Hiervanaf kan je gaan verkennen.

Goal

FPL should maximize the capacity it affords its operators to achieve their goals per unit of complexity and effort.

Those goals: solving problems by making fast, efficient, safe and ergonomic software.

What FPL is

A two-level language (2LTT [1]): a fixed meta level over an object level whose theory is pluggable. The default object theory is call-by-push-value (CBPV [2]). Terms are abstract binding trees (ABT [3]); the text is homoiconic and concatenative.

Get started

These entry points are planned, not yet published.

Try it

```
nix run github:fpl-org/fpl   # try FPL without cloning
```

In a container

```
podman run -it ghcr.io/fpl-org/fpl
```

Hack on it

```
git clone https://github.com/fpl-org/fpl && cd fpl
direnv allow       # once; without direnv, run `nix develop` each time you enter
nix flake check    # run every gate
```

Where things are

References

1. Annenkov, Capriotti, Kraus, Sattler, *Two-level type theory and applications* (MSCS, 2023), open copy; Kovács, *Staged compilation with two-level type theory* (ICFP 2022), open copy.
2. Levy, *Call-by-Push-Value* (Springer, 2003), open copy (PhD thesis).
3. Harper, *Practical Foundations for Programming Languages* (2nd ed., Cambridge, 2016), ch. 1, open copy (abbreviated edition).

Try it in Marimo

```
fpl.run.default.file("03-fpl/03.01-features/draft2/examples/01-frames.fpl")
```

```
'2 3 4\n[ 1 2 3 + ]\n5\n0 1 2\n'
```

```
fpl.run.default.src("""
double : n -- n
    dup +

2 double
""")
```

```
'4\n'
```

```
fpl.run.default.src("""
4 double
""")
```

```
UnresolvedError: ERROR: 2:3 unknown word: double
---------------------------------------------------------------------------
UnresolvedError                           Traceback (most recent call last)
Cell In[4], line 1
----> 1 fpl.run.default.src("""
      2 4 double
      3 """)
File ~/dev/github.com/psjg/fpl/worktrees/literate/01-bootstrap/python/fpl/run.py:73, in Sessi
     69 def src(self, source: str) -> str:
     70     """
     71     Render a source string.
     72     """
---> 73     return _run(source, self.name)
File ~/dev/github.com/psjg/fpl/worktrees/literate/01-bootstrap/python/fpl/run.py:51, in _run
     47 """
     48 Render one program. `session` is accepted and not stored.
     49 """
     50 del session
---> 51 return render(listing(run_now(_notebook_indent(source))))
File ~/dev/github.com/psjg/fpl/worktrees/literate/01-bootstrap/python/fpl/arrow.py:24, in Pas
     23 def __call__(self, value: A) -> B:
---> 24     return self.fn(value)
File ~/dev/github.com/psjg/fpl/worktrees/literate/01-bootstrap/python/fpl/arrow.py:27, in Pas
     26 def __rshift__[C](self, nxt: Pass[B, C]) -> Pass[A, C]:
---> 27     return Pass(f"{self.name}>>{nxt.name}", lambda value: nxt(self(value)))
File ~/dev/github.com/psjg/fpl/worktrees/literate/01-bootstrap/python/fpl/arrow.py:24, in Pas
     23 def __call__(self, value: A) -> B:
---> 24     return self.fn(value)
File ~/dev/github.com/psjg/fpl/worktrees/literate/01-bootstrap/python/fpl/arrow.py:27, in Pas
     26 def __rshift__[C](self, nxt: Pass[B, C]) -> Pass[A, C]:
---> 27     return Pass(f"{self.name}>>{nxt.name}", lambda value: nxt(self(value)))
File ~/dev/github.com/psjg/fpl/worktrees/literate/01-bootstrap/python/fpl/arrow.py:24, in Pas
     23 def __call__(self, value: A) -> B:
---> 24     return self.fn(value)
File ~/dev/github.com/psjg/fpl/worktrees/literate/01-bootstrap/python/fpl/arrow.py:27, in Pas
     26 def __rshift__[C](self, nxt: Pass[B, C]) -> Pass[A, C]:
---> 27     return Pass(f"{self.name}>>{nxt.name}", lambda value: nxt(self(value)))
File ~/dev/github.com/psjg/fpl/worktrees/literate/01-bootstrap/python/fpl/arrow.py:24, in Pas
     23 def __call__(self, value: A) -> B:
---> 24     return self.fn(value)
File ~/dev/github.com/psjg/fpl/worktrees/literate/01-bootstrap/python/fpl/desugar/__init__.py
    153         if isinstance(entry, Mount):
    154             catalog.directory(here, entry.name)
```

```
--> 155 return tuple(
    156     _Desugar(catalog, trivia.docstrings(program)).statements(lines, ())
    157 )
File ~/dev/github.com/psjg/fpl/worktrees/literate/01-bootstrap/python/fpl/desugar/__init__.py
    859     yield from self.statements(coded(line.block), (*here, name))
    860 elif not here:
--> 861     yield Run(self.body((line,), 0, here), line.span)
File ~/dev/github.com/psjg/fpl/worktrees/literate/01-bootstrap/python/fpl/desugar/__init__.py
   1042     balance = 0
   1043     continue
   1044 parts = (
   1045     *(
   1046         partial(self.child, child, kind)
-> 1047         for child, kind in self.filled(line)
   1048     ),
   1049     *(partial(self.frame, frame) for frame in line.frames),
   1050 )
   1051 for part in parts:
   1052     before = self.effects
File ~/dev/github.com/psjg/fpl/worktrees/literate/01-bootstrap/python/fpl/desugar/__init__.py
   1258 block = coded(line.block)
   1259 outer = self.effects
-> 1260 inputs = self.inputs(self.routed(self.frames(line.frames), len(block)))
   1261 self.effects = outer
   1262 kinds: list[Slot | None] = [None] * len(block)
File ~/dev/github.com/psjg/fpl/worktrees/literate/01-bootstrap/python/fpl/desugar/__init__.py
   1304 def frames(self, frames: tuple[Frame, ...]) -> tuple[Node, ...]:
   1305     """
   1306     Frames joined, with no saturation.
   1307     """
-> 1308     return tuple(node for frame in frames for node in self.frame(frame))
File ~/dev/github.com/psjg/fpl/worktrees/literate/01-bootstrap/python/fpl/desugar/__init__.py
   1304 def frames(self, frames: tuple[Frame, ...]) -> tuple[Node, ...]:
   1305     """
   1306     Frames joined, with no saturation.
   1307     """
-> 1308     return tuple(node for frame in frames for node in self.frame(frame))
File ~/dev/github.com/psjg/fpl/worktrees/literate/01-bootstrap/python/fpl/desugar/__init__.py
   1319             code.append(Push(Strand(tuple(map(atom, items)))))
   1320         else:
-> 1321             code.extend(
   1322                 node for item in items for node in self.item(item)
   1323             )
```

```
   1324 return tuple(code)
File ~/dev/github.com/psjg/fpl/worktrees/literate/01-bootstrap/python/fpl/desugar/__init__.py
   1319             code.append(Push(Strand(tuple(map(atom, items)))))
   1320         else:
   1321             code.extend(
-> 1322                 node for item in items for node in self.item(item)
   1323             )
   1324 return tuple(code)
File ~/dev/github.com/psjg/fpl/worktrees/literate/01-bootstrap/python/fpl/desugar/__init__.py
   1330 match item:
   1331     case Word():
-> 1332         return (self.word(item),)
   1333     case Text():
   1334         return (Push(text(item)),)
File ~/dev/github.com/psjg/fpl/worktrees/literate/01-bootstrap/python/fpl/desugar/__init__.py
   1354     self.effects = self.effects.new_child({name: LOCAL})
   1355     return Bind(name, (), word.span)
-> 1356 return self.call(word)
File ~/dev/github.com/psjg/fpl/worktrees/literate/01-bootstrap/python/fpl/desugar/__init__.py
   1369 found = self.catalog.find(here, word.body)
   1370 if found is None:
-> 1371     raise missing(word, self.catalog.listed(here, word.body))
   1372 if found in self.catalog.refused:
   1373     raise FplError(word.span, self.catalog.refused[found])
UnresolvedError: ERROR: 2:3 unknown word: double
```

1 Bootstrap

01.01.01 Source

01.01.01 Source

The pipe does not start on FON. This is the ingress.

```
import sys

from fpl.pipe import cli


def main() -> None:
    sys.exit(cli("source"))


if __name__ == "__main__":
    main()
```

01.01.02 FON

01.01.02 FON

This is the seed's FON reader and writer.

```
"""
FON data: the inert, braced data profile of FPL, read within bounds and
written canonically.

The reader never evaluates, resolves or constructs typed objects: a bare
name or #name is a symbol, $name or $<code>:<n> a reference by identity,
&<multihash> a reference by content (fpl/multihash.py, the hex alias
read too), [ ] opaque code, ( ) a tagged tuple, { } a dict with literal
keys, a number an exact decimal and _ absence. Whitespace separates
tokens only, in any amount; there are no comments, blocks or islands.
Strings are counted by the code grammar's pre-lexer. Depth, token length
and input size are parameters.
"""

import contextlib
import math
import re
import struct
from collections.abc import Callable
from dataclasses import dataclass
from decimal import Decimal
from functools import cache
from typing import assert_never

from lark import Lark, Token, Tree, UnexpectedInput

from fpl.arrow import Pass
from fpl.errors import FplError, Span
from fpl.lex import Lines, Prelexed, pairs
from fpl.multihash import Multihash, spelling

GRAMMAR = r"""
start: value*
```

```
?value: TOKEN | list | dict | tuple | code
list:  " " value* " "
dict:  "{" (TOKEN value)* "}"
tuple: "(" value* ")"
code:  "[" value* "]"
TOKEN: /[^\s\[\]()"" ⟦⟧ {}| ]+/
%ignore /\s+/
"""
STRINGS = re.compile(r"[" ⟦" ⟧]")
OPENERS = {"[": "]", "(": ")", " ": " ", "{": "}"}
NUMBER = re.compile(r"-?(0|[1-9][0-9]*)(\.[0-9]+)?")
NUMERIC = re.compile(r"-?(0x|[0-9])")
HEXFLT = re.compile(r"-?0x(0|1)(\.[0-9a-f]*[1-9a-f])?p-?(0|[1-9][0-9]*)")
TOKEN = re.compile(r"[^\s\[\]()"" ⟦⟧ {}| ¶]+")


@dataclass(frozen=True)
class Absent:
    """
    _ : nothing here.
    """

    def written(self) -> str:
        """
        Canonical FON.
        """
        return "_"


@dataclass(frozen=True)
class Bool:
    """
    true or false, a kind apart from numbers.
    """

    value: bool

    def written(self) -> str:
        """
        Canonical FON.
        """
        return "true" if self.value else "false"
```

```
@dataclass(frozen=True)
class Num:
    """
    An exact decimal: no +, no leading zeros, no exponent.
    """

    value: Decimal

    def written(self) -> str:
        """
        Canonical FON.
        """
        return format(self.value, "f")


@dataclass(frozen=True)
class Flt:
    """
    An IEEE binary64 float, held as its bit pattern so that every float,
    nan and -0.0 included, equals only itself; written as C99 %a hex,
    exact.
    """

    bits: int

    @classmethod
    def of(cls, value: float) -> "Flt":
        """
        The float's bit pattern.
        """
        return cls(int(struct.unpack("<q", struct.pack("<d", value))[0]))

    @property
    def value(self) -> float:
        """
        The float the bits spell.
        """
        return float(struct.unpack("<d", struct.pack("<q", self.bits))[0])

    def written(self) -> str:
        """
        Canonical FON: nan, -nan, ω, -ω, [-]0x0p0, or [-]0x1[.hex]p<exp>
```

```
        with trailing zero hex digits dropped. A subnormal is normalised
        like every other nonzero float, its exponent below -1022: the
        smallest is 0x1p-1074, not float.hex's 0x0.0000000000001p-1022,
        so that each float has one spelling.
        """
        x = self.value
        sign = "-" if math.copysign(1, x) < 0 else ""
        if math.isnan(x):
            return sign + "nan"
        if math.isinf(x):
            return sign + "ω"
        if x == 0:
            return sign + "0x0p0"
        fraction, biased = (
            self.bits & ((1 << 52) - 1),
            (self.bits >> 52) & 0x7FF,
        )
        exponent = biased - 1023
        if biased == 0:
            top = fraction.bit_length() - 1
            fraction, exponent = (
                (fraction - (1 << top)) << (52 - top),
                top - 1074,
            )
        digits = f"{fraction:013x}".rstrip("0")
        return f"{sign}0x1{'.' if digits else ''}{digits}p{exponent}"


@dataclass(frozen=True)
class Sym:
    """
    A symbol: a bare name, or #name; not resolved.
    """

    name: str

    def written(self) -> str:
        """
        Canonical FON.
        """
        return self.name
```

```
@dataclass(frozen=True)
class Cell:
    """
    $name, or $<code>:<n> naming by hash: a reference by identity; not
    followed.
    """

    name: str | Multihash

    def written(self) -> str:
        """
        Canonical FON.
        """
        return "$" + (
            self.name.spelled()
            if isinstance(self.name, Multihash)
            else self.name
        )


@dataclass(frozen=True)
class Hash:
    """
    &<multihash>: a reference by content, the bytes themselves up to 32;
    not followed.
    """

    ref: Multihash

    def written(self) -> str:
        """
        Canonical FON: the decimal spelling.
        """
        return "&" + self.ref.spelled()


@dataclass(frozen=True)
class Str:
    """
    "…" text, interpolation kept as text, or … raw text.
    """

    text: str
```

```
    raw: bool = False

    def written(self) -> str:
        """
        Canonical FON.
        """
        return f" {self.text} " if self.raw else f""{self.text}""


@dataclass(frozen=True)
class List:
    """
      …  values.
    """

    items: tuple["Value", ...]

    def written(self) -> str:
        """
        Canonical FON; the empty list is  .
        """
        return f"  {_spaced(self.items)}  " if self.items else " "


@dataclass(frozen=True)
class Tagged:
    """
    ( … ): a tagged tuple; its head is data, never called.
    """

    items: tuple["Value", ...]

    def written(self) -> str:
        """
        Canonical FON.
        """
        return f"( {_spaced(self.items)} )" if self.items else "( )"


@dataclass(frozen=True)
class Code:
    """
    [ … ]: code, opaque to the reader.
```

```
    """

    items: tuple["Value", ...]

    def written(self) -> str:
        """
        Canonical FON.
        """
        return f"[ {_spaced(self.items)} ]" if self.items else "[ ]"


type Leaf = Absent | Bool | Num | Flt | Sym | Cell | Hash | Str


@dataclass(frozen=True)
class Dict:
    """
    { key value … }: keys literal and distinct, in the order written.
    """

    entries: tuple[tuple[Leaf, "Value"], ...]

    def written(self) -> str:
        """
        Canonical FON.
        """
        body = " ".join(
            f"{key.written()} {value.written()}" for key, value in self.entries
        )
        return f"{{ {body} }}" if self.entries else "{ }"


type Value = Leaf | List | Tagged | Code | Dict
type Json = bool | int | float | str | list["Json"] | dict[str, "Json"] | None
SEQUENCES: dict[str, Callable[[tuple[Value, ...]], Value]] = {
    "list": List,
    "tuple": Tagged,
    "code": Code,
}

CONSTANTS: dict[str, Leaf] = {
    "_": Absent(),
    "true": Bool(True),
```

```
    "false": Bool(False),
    "ω": Flt.of(math.inf),
    "-ω": Flt.of(-math.inf),
    "nan": Flt.of(math.nan),
    "-nan": Flt.of(-math.nan),
}


def _spaced(values: tuple[Value, ...]) -> str:
    """
    Values separated by one space.
    """
    return " ".join(value.written() for value in values)


class _RefusedError(Exception):
    """
    A token the profile refuses, before it is placed in the source.
    """


def _hash(text: str) -> Hash:
    """
    &<multihash>, in decimal or the hex alias, inline bytes allowed.
    """
    try:
        return Hash(spelling(text, inline=True))
    except ValueError:
        raise _RefusedError(f"malformed hash reference &{text}") from None


def _cell(text: str) -> Cell:
    """
    $name, or $<code>:<n> when it opens with a digit; no hex alias,
    which would read as a name, and no inline bytes, which are content,
    not identity.
    """
    if text[0] not in "0123456789":
        return Cell(text)
    if ":" in text:
        with contextlib.suppress(ValueError):
            return Cell(spelling(text, inline=False))
    raise _RefusedError(f"malformed identity ${text}")
```

```
def _hexflt(token: str) -> Flt:
    """
    A hex float, read only in the spelling Flt.written gives it: a
    second spelling of the same bits (0x0.8p0 for 0x1p-1, 0x0p5 for
    0x0p0) or one that rounds or underflows (0x1p-2000) is refused, so
    that write  read is the identity on what reads, as it is for
    decimals; past binary64 (0x1p2000) is out of range.
    """
    try:
        flt = Flt.of(float.fromhex(token))
    except OverflowError:
        raise _RefusedError(f"float out of range {token}") from None
    if flt.written() != token:
        raise _RefusedError(f"non-canonical number {token}")
    return flt


SIGILS: dict[str, Callable[[str], Leaf]] = {"$": _cell, "&": _hash, "#": Sym}


def leaf(token: str) -> Leaf:
    """
    The value one token spells; a number or hash out of canonical form
    is refused.
    """
    constant = CONSTANTS.get(token)
    if constant is not None:
        return constant
    if HEXFLT.fullmatch(token):
        return _hexflt(token)
    if NUMBER.fullmatch(token):
        return Num(Decimal(token))
    if NUMERIC.match(token):
        raise _RefusedError(f"non-canonical number {token}")
    sigil = SIGILS.get(token[0]) if len(token) > 1 else None
    return Sym(token) if sigil is None else sigil(token[1:])


@cache
def parser() -> Lark:
    """
```

```
    The LALR parser of the braced profile.
    """
    return Lark(GRAMMAR, parser="lalr")


class _Build:
    """
    Values from Lark's tree, every refusal placed in the source.
    """

    def __init__(self, text: str, lexed: Prelexed, max_token: int) -> None:
        self.text, self.lexed, self.max_token = text, lexed, max_token

    def at(self, offset: int) -> Span:
        """
        The source position of a code offset.
        """
        return Lines(self.text).span(self.lexed.origin[offset])

    def leaf(self, token: Token) -> Leaf:
        """
        A token's value: a stashed string, or what the token spells.
        """
        start = token.start_pos or 0
        stashed = self.lexed.stash.get(start)
        if stashed is not None:
            return Str(
                self.text[stashed.start : stashed.end], stashed.kind == "raw"
            )
        if len(token) > self.max_token:
            raise FplError(self.at(start), "token too long")
        try:
            return leaf(str(token))
        except _RefusedError as refusal:
            raise FplError(self.at(start), str(refusal)) from None

    def value(self, node: Tree[Token] | Token) -> Value:
        """
        A value from a token or an enclosure.
        """
        if isinstance(node, Token):
            return self.leaf(node)
        if node.data == "dict":
```

```
            return self.dict(node.children)
        items = tuple(self.value(child) for child in node.children)
        return SEQUENCES[str(node.data)](items)

    def dict(self, children: list[Tree[Token] | Token]) -> Dict:
        """
        Keys and values in turn; a key given twice is refused where it
        repeats.
        """
        entries: dict[Leaf, Value] = {}
        for key_token, node in zip(children[::2], children[1::2], strict=True):
            assert isinstance(key_token, Token)
            key = self.leaf(key_token)
            if key in entries:
                raise FplError(
                    self.at(key_token.start_pos or 0),
                    f"repeated key {key.written()}",
                )
            entries[key] = self.value(node)
        return Dict(tuple(entries.items()))


def _depth(text: str, lexed: Prelexed, max_depth: int) -> None:
    """
    Refuse an enclosure opened deeper than max_depth, or one never
    closed, by a linear scan of the code; a closer out of turn is left
    to the grammar.
    """
    open_: list[int] = []
    for offset, char in enumerate(lexed.code):
        if char in OPENERS:
            open_.append(offset)
        elif open_ and char == OPENERS[lexed.code[open_[-1]]]:
            open_.pop()
        if len(open_) > max_depth:
            raise FplError(
                Lines(text).span(lexed.origin[offset]), "nesting too deep"
            )
    if open_:
        char = lexed.code[open_[-1]]
        raise FplError(
            Lines(text).span(lexed.origin[open_[-1]]), f"{char} never closed"
        )
```

```
def _placed(lines: Lines, lexed: Prelexed, offset: int | None) -> Span:
    """
    The source position of Lark's failure; an unknown or end position is
    the text's end.
    """
    return lines.span(
        lexed.origin[offset if offset is not None and offset >= 0 else -1]
    )


def read(
    text: str,
    max_depth: int = 64,
    max_token: int = 4096,
    max_size: int = 1 << 24,
) -> Value:
    """
    The value a FON document spells: one value is itself, several or
    none a List of them. Linear and total: every refusal is an FplError
    inside the text, bounds included.
    """
    lines = Lines(text)
    if len(text) > max_size:
        raise FplError(lines.span(max_size), "input too large")
    lexed = pairs(text, special=STRINGS)
    _depth(text, lexed, max_depth)
    try:
        tree = parser().parse(lexed.code)  # pyright: ignore[reportUnknownMemberType] -- lar
    except UnexpectedInput as failure:
        raise FplError(
            _placed(lines, lexed, failure.pos_in_stream), "unexpected input"
        ) from None
    values = tuple(
        _Build(text, lexed, max_token).value(child) for child in tree.children
    )
    return values[0] if len(values) == 1 else List(values)


def write(value: Value) -> str:
    """
    Canonical FON for a value, the text `read` gives it back from. A
```

```
    text holding an unbalanced " or  , or a symbol spelling another
    kind, has no FON spelling (HOLES.md: fon-unwritable).
    """
    return value.written()


def embed(json: Json) -> Value:
    """
    A JSON value as FON: null is absence, a number an exact decimal, an
    object a Dict keyed by symbols where the key reads back as one, else
    by text.
    """
    match json:
        case list():
            return List(tuple(embed(item) for item in json))
        case dict():
            return Dict(
                tuple((_key(key), embed(item)) for key, item in json.items())
            )
        case _:
            return _scalar(json)


def _scalar(json: bool | float | str | None) -> Leaf:
    """
    A JSON scalar as FON.
    """
    match json:
        case None:
            return Absent()
        case bool():
            return Bool(json)
        case int() | float():
            return Num(Decimal(repr(json)))
        case str():
            return Str(json)
        case _:
            assert_never(json)


def _key(name: str) -> Leaf:
    """
    A JSON key as a symbol when it spells one, else as text.
```

```
    """
    return (
        Sym(name)
        if TOKEN.fullmatch(name) and _spells(name) == Sym(name)
        else Str(name)
    )


def _spells(token: str) -> Leaf | None:
    """
    What a token reads as, or None when refused.
    """
    try:
        return leaf(token)
    except _RefusedError:
        return None


fon = Pass("fon", read)
```

01.01.03 Pipe

01.01.03 Pipe

`fonify` is the whole trip to the wire. `source` only fonifies. `render` only defonifies.

```
import sys
from decimal import Decimal

from fpl.arrow import Pass
from fpl.errors import FplError
from fpl.fon import Dict, List, Num, Str, Tagged, read, write
from fpl.lex import Prelexed, Stashed, Tokens, lex
from fpl.parse import parse

stdio = Pass("stdio", lambda stage: stage)


def _text(value: object) -> str:
    if not isinstance(value, Str):
        raise FplError(
            __import__("fpl.errors", fromlist=["Span"]).Span(1, 1),
            "expected text",
        )
    return value.text


def fonify_text(text: str) -> bytes:
    """
    A → bytes. Raw quotes when the text allows them.
    """
    raw = " " not in text and " " not in text
    return (write(Str(text, raw)) + "\n").encode()


def defonify_text(data: bytes) -> str:
    got = read(data.decode())
    if not isinstance(got, Str):
        raise ValueError("expected a FON string")
```

```
    return got.text


def _num(value: int) -> Num:
    return Num(Decimal(value))


def fonify_tokens(tokens: Tokens) -> bytes:
    stash = tuple(
        Tagged(
            (
                _num(key),
                _num(item.start),
                _num(item.end),
                Str(item.kind, raw=True),
                _num(item.depth),
            )
        )
        for key, item in tokens.lexed.stash.items()
    )
    raw = tuple(
        Tagged((_text_sym(kind), Str(value, raw=True), _num(start), _num(end)))
        for kind, value, start, end in tokens.raw
    )
    document = Dict(
        (
            (Str("source", raw=True), Str(tokens.source, raw=True)),
            (Str("code", raw=True), Str(tokens.lexed.code, raw=True)),
            (
                Str("origin", raw=True),
                List(tuple(_num(n) for n in tokens.lexed.origin)),
            ),
            (Str("stash", raw=True), List(stash)),
            (Str("tokens", raw=True), List(raw)),
        )
    )
    return (write(document) + "\n").encode()


def _text_sym(kind: str) -> Str:
    return Str(kind, raw=True)
```

```
def _pair(item: object) -> tuple:
    if not isinstance(item, Tagged):
        raise ValueError("expected a tagged tuple")
    return item.items


def defonify_tokens(data: bytes) -> Tokens:
    got = read(data.decode())
    if not isinstance(got, Dict):
        raise ValueError("expected a FON dict")
    fields = {
        key.text: value for key, value in got.entries if isinstance(key, Str)
    }
    origin = fields["origin"]
    stash = fields["stash"]
    raw = fields["tokens"]
    if (
        not isinstance(origin, List)
        or not isinstance(stash, List)
        or not isinstance(raw, List)
    ):
        raise ValueError("malformed tokens")
    held: dict[int, Stashed] = {}
    for item in stash.items:
        key, start, end, kind, depth = _pair(item)
        held[int(key.value)] = Stashed(
            kind.text, int(start.value), int(end.value), int(depth.value)
        )
    lexed = Prelexed(
        _text(fields["code"]), tuple(int(n.value) for n in origin.items), held
    )
    packed = tuple(
        (kind.text, value.text, int(start.value), int(end.value))
        for kind, value, start, end in (_pair(item) for item in raw.items)
    )
    return Tokens(_text(fields["source"]), lexed, packed)


def cli(stage: object) -> int:
    """
    Read stdin, run one stage, write stdout. A Pass or its name both
    work.
    """
```

```
    name = stage if isinstance(stage, str) else stage.name
    data = sys.stdin.buffer.read()
    try:
        sys.stdout.buffer.write(_run(name, data))
    except (FplError, ValueError) as error:
        print(error, file=sys.stderr)
        return 1
    return 0


def _run(name: str, data: bytes) -> bytes:
    if name == "source":
        return fonify_text(data.decode())
    if name == "render":
        return defonify_text(data).encode()
    if name == "lex":
        return fonify_tokens(lex(defonify_text(data)))
    text = defonify_text(data) if name != "parse" else ""
    if name == "parse":
        program = parse(defonify_tokens(data))
        from fpl.print import render

        return fonify_text(render(program))
    from fpl.desugar import desugar
    from fpl.elaborate import elaborate
    from fpl.lex import lex as lex_pass
    from fpl.parse import parse as parse_pass

    program = parse_pass(lex_pass(text))
    if name == "desugar":
        desugar(program)
        from fpl.print import render

        return fonify_text(render(program))
    statements = desugar(program)
    if name == "elaborate":
        elaborate(statements)
        from fpl.print import render

        return fonify_text(render(program))
    if name == "eval":
        from fpl.desugar import listing
        from fpl.evaluate import evaluate
```

```
    from fpl.print import render

    return fonify_text(render(listing(evaluate(elaborate(statements)))))
raise ValueError(f"unknown stage {name}")
```

01.01.04 Render

01.01.04 Render

The pipe ends in text a person can read.

```
import sys

from fpl.pipe import cli


def main() -> None:
    sys.exit(cli("render"))


if __name__ == "__main__":
    main()
```

01.02.01 Lex

01.02.01 Lex

The hand counter in the seed's `lex.py` counts pairs and tabs, because a grammar cannot. Then a Lark lexer, and nothing else, yields the tokens.

01.02.02-parse reads those tokens with a second Lark.

```
"""
The pre-lexer: what the grammar cannot count, counted before Lark sees
the text.

Strings " " and    and block comments ⟦ ⟧ are pairs counted by their
own pair only, so they cannot overlap. A string becomes one placeholder
token, its interior left in the source for the pair's reader; a block
comment becomes nothing. Indentation is then held strict: tabs only, and
no bracket left open across a dent below the line that opened it. The
affix pass reads a token's shape after lexing. Positions stay in the
source: every code character keeps its origin.
"""

import re
from bisect import bisect_right
from collections.abc import Callable
from dataclasses import dataclass
from pathlib import Path
from typing import Literal

from lark import Lark

from fpl.arrow import Pass
from fpl.ast_surface import Kind
from fpl.errors import FplError, Span

PAIRS = {""": """, " ": " ", "⟦": "⟧"}
BRACKETS = {"[": "]", "(": ")", " ": " ", "{": "}"}
CLOSERS = {close: opener for opener, close in (PAIRS | BRACKETS).items()}
PLACEHOLDER = "¶"
SPECIAL = re.compile(r"[" ⟦" ⟧; ]")
```

```
LINE_COMMENT = re.compile(r"(;+| )[^\n]*")
MODS = "\u00b4`\u00a8\u02dc\u207c\u02d9"
SHAPE = re.compile(rf"(->|→|\$|#|&|\.\./|\.\.)?([^{MODS}]*?)([{MODS}]*)")
NUMBER = re.compile(r"-?(\d+(\.\d+)?|ϖ| )")


class Lines:
    """
    Offsets into a source as the 1-based line and column an editor
    shows.
    """

    def __init__(self, source: str) -> None:
        self.starts = [0] + [
            n + 1 for n, char in enumerate(source) if char == "\n"
        ]

    def span(self, offset: int) -> Span:
        """
        The line and column of a source offset; the end is just past the
        last line.
        """
        line = bisect_right(self.starts, offset)
        return Span(line, offset - self.starts[line - 1] + 1)


@dataclass(frozen=True)
class Stashed:
    """
    A string's interior, [start, end) in the source: `str` interpolates
      , `raw` does not. depth is the tabs before the line it opens on,
    which its continuation lines drop (S26).
    """

    kind: Literal["str", "raw"]
    start: int
    end: int
    depth: int


@dataclass(frozen=True)
class Prelexed:
    """
```

```
    Code for the grammar, the source offset of each of its characters
    and of its end, and the string each placeholder stands for, by the
    placeholder's offset in the code.
    """

    code: str
    origin: tuple[int, ...]
    stash: dict[int, Stashed]


def counted(text: str, at: int, close: str, end: int) -> int | None:
    """
    The offset just past the closer that balances the opener at `at`, or
    None if none does before `end`. Only the opener's own pair is
    counted.
    """
    opener, depth, j = text[at], 1, at + 1
    while j < end and depth:
        depth += (text[j] == opener) - (text[j] == close)
        j += 1
    return None if depth else j


class _Code:
    """
    The code being written, each character with its source offset.
    """

    def __init__(self) -> None:
        self.chars: list[str] = []
        self.origin: list[int] = []
        self.stash: dict[int, Stashed] = {}

    def put(self, text: str, at: int, verbatim: bool = True) -> None:
        """
        Append text from source offset `at`; a pad or placeholder keeps
        `at` for every char.
        """
        self.chars.extend(text)
        self.origin.extend(
            at + k if verbatim else at for k in range(len(text))
        )
```

```
    def at_line_start(self) -> bool:
        """
        Nothing but tabs written since the last newline.
        """
        for char in reversed(self.chars):
            if char != "\t":
                return char == "\n"
        return True


def _pair(source: str, at: int, end: int, code: _Code) -> int:
    """
    Write the string or comment opening at `at`; return the offset after
    its closer.
    """
    opener = source[at]
    after = counted(source, at, PAIRS[opener], end)
    if after is None:
        raise FplError(Lines(source).span(at), f"{opener} never closed")
    if opener == "[":
        if code.at_line_start():
            return (
                len(source[after:end])
                - len(source[after:end].lstrip(" "))
                + after
            )
        code.put(" ", at)
        return after
    if code.chars and code.chars[-1] not in " \t\n":
        code.put(" ", at)
    line = source.rfind("\n", 0, at) + 1
    depth = len(source[line:at]) - len(source[line:at].lstrip("\t"))
    code.stash[len(code.chars)] = Stashed(
        "str" if opener == """ else "raw", at + 1, after - 1, depth
    )
    code.put(PLACEHOLDER + " ", at, verbatim=False)
    return after


def _special(source: str, at: int, end: int, code: _Code) -> int:
    """
    Write what starts with a special character at `at`; return where the
    next write starts.
```

```
    """
    char = source[at]
    if char in PAIRS:
        return _pair(source, at, end, code)
    if char in CLOSERS:
        raise FplError(Lines(source).span(at), f"{char} closes nothing")
    newline = source.find("\n", at, end)
    after = end if newline < 0 else newline
    code.put(source[at:after], at)
    return after


def pairs(
    source: str,
    start: int = 0,
    end: int | None = None,
    special: re.Pattern[str] = SPECIAL,
) -> Prelexed:
    """
    Stash the strings and drop the block comments of source[start:end],
    ending the code in a newline; `special` names the characters that
    open or close a pair or a line comment. Refuses a pair never closed
    and a closer with no opener; indentation is not looked at.
    """
    stop = len(source) if end is None else end
    code, at = _Code(), start
    while at < stop:
        found = special.search(source, at, stop)
        upto = found.start() if found else stop
        code.put(source[at:upto], at)
        at = _special(source, upto, stop, code) if found else stop
    code.put("\n", stop)
    return Prelexed("".join(code.chars), (*code.origin, stop), code.stash)


def prelex(source: str, start: int = 0, end: int | None = None) -> Prelexed:
    """
    Pre-lex source[start:end] (all of it by default) to code the grammar
    reads, ending in a newline. Refuses a pair never closed, a closer
    with no opener, and loose indentation.
    """
    lexed = pairs(source, start, end)
    strict(source, lexed.code, lexed.origin)
```

```
    return lexed


class _RefusalError(Exception):
    """
    A strictness refusal at a code offset, before it is placed in the
    source.
    """

    def __init__(self, offset: int, message: str) -> None:
        super().__init__(message)
        self.offset = offset
        self.message = message


def _brackets(
    line: str, base: int, depth: int, open_: list[tuple[str, int, int]]
) -> None:
    """
    Track the brackets of one code line, its line comment aside, on the
    stack of open ones; refuse a closer that closes nothing or not the
    latest opener. Offsets are into the code.
    """
    body = LINE_COMMENT.split(line, maxsplit=1)[0]
    for n, char in enumerate(body):
        if char in BRACKETS:
            open_.append((char, base + n, depth))
        elif char in CLOSERS and not open_:
            raise _RefusalError(base + n, f"{char} closes nothing")
        elif char in CLOSERS and open_[-1][0] != CLOSERS[char]:
            raise _RefusalError(
                base + n, f"{char} does not close {open_[-1][0]}"
            )
        elif char in CLOSERS:
            open_.pop()


def _lines(code: str, line_of: Callable[[int], int]) -> None:
    """
    Refuse leading spaces outside brackets, and a dedent below the line
    of a bracket still open; the dedent is reported at the dedenting
    line and names the opener's line.
    """
```

```
    open_: list[tuple[str, int, int]] = []
    base = 0
    for line in code.split("\n"):
        depth = len(line) - len(line.lstrip("\t"))
        if not open_ and line[depth : depth + 1] == " ":
            raise _RefusalError(base + depth, "indentation must be tabs")
        if open_ and line.strip() and depth < open_[-1][2]:
            opener, opened, _ = open_[-1]
            message = (
                f"dedent below line {line_of(opened)}, "
                f"but its {opener} is still open"
            )
            raise _RefusalError(base + depth, message)
        _brackets(line, base, depth, open_)
        base += len(line) + 1
    if open_:
        raise _RefusalError(open_[-1][1], f"{open_[-1][0]} never closed")


def strict(source: str, code: str, origin: tuple[int, ...]) -> None:
    """
    Hold the code to strict indentation, reporting a refusal at its
    place in the source.
    """
    lines = Lines(source)
    try:
        _lines(code, lambda offset: lines.span(origin[offset]).line)
    except _RefusalError as refusal:
        raise FplError(
            lines.span(origin[refusal.offset]), refusal.message
        ) from None


def shape(token: str) -> tuple[str, Kind, tuple[str, ...], str] | None:
    """
    A token's prefix sigil, kind, body (a path's segments) and suffix
    modifiers, or None when a modifier stands anywhere but at the end.
    """
    match = SHAPE.fullmatch(token)
    if match is None:
        return None
    prefix, body, mods = match.group(1) or "", match.group(2), match.group(3)
    kind: Kind = (
```

```
        "modifier"
        if not body
        else "number"
        if NUMBER.fullmatch(body)
        else "path"
        if "/" in body
        else "name"
    )
    return (
        prefix,
        kind,
        tuple(body.split("/")) if kind == "path" else (body,),
        mods,
    )


GRAMMAR = Path(__file__).resolve().parents[1] / "grammar.lark"


@dataclass(frozen=True)
class Tokens:
    """
    The lexer's output: the pre-lexed text and the tokens the second
    Lark will read.
    """

    source: str
    lexed: Prelexed
    raw: tuple[tuple[str, str, int, int], ...]


_LEXER: Lark | None = None


def token_lark() -> Lark:
    """
    The lexer Lark. It does not parse.
    """
    global _LEXER
    if _LEXER is None:
        _LEXER = Lark(
            GRAMMAR.read_text(),
            parser="lalr",
```

```
            propagate_positions=True,
            maybe_placeholders=False,
        )
    return _LEXER


def code_tokens(code: str) -> list:
    """
    Tokens for code the pre-lexer already produced.
    """
    return list(token_lark().lex(code))


def _pack(got: list) -> tuple[tuple[str, str, int, int], ...]:
    return tuple(
        (token.type, str(token), token.start_pos or 0, token.end_pos or 0)
        for token in got
    )


def _lex(source: str) -> Tokens:
    lexed = prelex(source)
    return Tokens(source, lexed, _pack(code_tokens(lexed.code)))


lex = Pass("lex", _lex)
```

01.02.02 Parse

01.02.02 Parse

Islands inside strings are new source, so those still go through the lexer Lark. The program itself is the token stream from 01.02.01-lex.

```
"""
Source text to the surface AST: pre-lex, the tab indenter, LALR over
fpl/grammar.lark, then the affix pass on every token. No semantics live
here (.agents/CONVENTIONS.md).
"""

# pyright: reportIncompatibleMethodOverride=false, reportAssignmentType=false
# ruff: noqa: RUF012 -- Lark reads a class attribute

from collections.abc import Callable, Iterator
from functools import cache
from pathlib import Path
from typing import override

import icontract
from lark import Lark, Token, Tree, UnexpectedInput, UnexpectedToken
from lark.indenter import DedentError, Indenter
from lark.lexer import Lexer

from fpl.arrow import Pass
from fpl.ast_surface import (
    Cell,
    Comment,
    Enclosure,
    Frame,
    Item,
    Line,
    Pair,
    Program,
    Text,
    Word,
)
```

```
from fpl.errors import FplError, Span
from fpl.lex import (
    BRACKETS,
    Lines,
    Prelexed,
    Stashed,
    Tokens,
    code_tokens,
    counted,
    prelex,
    shape,
)

GRAMMAR = Path(__file__).resolve().parents[1] / "grammar.lark"
PAIRS: dict[str, Pair] = {
    "quotation": "quotation",
    "prefix": "prefix",
    "group": "group",
    "dict": "dict",
}
```

A source string becomes a Program. The second Lark reads the tokens lex produced and does not lex them again.

```
def parse(source: str) -> Program:
    """
    Parse source text to the surface AST. Every refusal, the
    pre-lexer's, the indenter's or the grammar's, is an FplError at a
    position inside the source; nothing else escapes. Nesting deeper
    than the build can recurse to is refused at the source's start.
    """
    try:
        return read(source, Lines(source), 0, len(source))
    except RecursionError:
        raise FplError(Span(1, 1), "nesting too deep to read") from None


def read(source: str, lines: Lines, start: int, end: int) -> Program:
    """
    Parse source[start:end], placing every node in the whole source.
    """
    lexed = prelex(source, start, end)
    tree = _tree(source, lexed)
```

```
    return _Build(source, lines, lexed).program(tree, lines.span(start))
```

The parser eats the tokens lex already made.

```
class _Replay(Lexer):
    """
    Yield tokens the lexer Lark already produced. This parser does not
    lex.
    """

    pending: list = []

    def __init__(self, lexer_conf: object) -> None:
        pass

    def lex(self, lexer_state: object, parser_state: object = None) -> object:
        yield from _Replay.pending


_REPLAY: Lark | None = None


def _replay_lark() -> Lark:
    global _REPLAY
    if _REPLAY is None:
        _REPLAY = Lark(
            GRAMMAR.read_text(),
            parser="lalr",
            lexer=_Replay,
            postlex=FplIndenter(),
            propagate_positions=True,
            maybe_placeholders=False,
        )
    return _REPLAY


def _from_tokens(raw: list) -> Tree[Token]:
    _Replay.pending = raw
    return _replay_lark().parse(" ")


def _as_tokens(raw: tuple[tuple[str, str, int, int], ...]) -> list[Token]:
    return [
```

```
        Token(kind, value, start_pos=start, end_pos=end)
        for kind, value, start, end in raw
    ]
```

The grammar does not count tabs. One tab is one level. A newline inside brackets is not a block.

```
class MisindentedError(Exception):
    """
    The indenter met a dedent to a level no line opened, at the newline
    token given.
    """

    def __init__(self, token: Token) -> None:
        super().__init__(token)
        self.token = token


class FplIndenter(Indenter):
    """
    One tab per level; a newline inside [ ] ( )    { } is not block
    structure.
    """

    # Lark declares these as abstract properties and then reads them as
    # class attributes. The two lists are class variables.
    NL_type = "_NL"
    OPEN_PAREN_types = ["LSQB", "LPAR", "_LANGLE", "_LBRACE"]
    CLOSE_PAREN_types = ["RSQB", "RPAR", "_RANGLE", "_RBRACE"]
    INDENT_type = "_INDENT"
    DEDENT_type = "_DEDENT"
    tab_len = 1

    @override
    def handle_NL(self, token: Token) -> Iterator[Token]:
        """
        Lark's indentation, with the newline where a dedent went wrong
        kept for the error.
        """
        try:
            yield from super().handle_NL(token)
        except DedentError as failure:
            raise MisindentedError(token) from failure
```

LALR over grammar.lark.

```
@cache
def parser() -> Lark:
    """
    The LALR parser, fed by the tab indenter.
    """
    return Lark(
        GRAMMAR.read_text(),
        parser="lalr",
        postlex=FplIndenter(),
        propagate_positions=True,
        maybe_placeholders=False,
    )


def _tree(source: str, lexed: Prelexed) -> Tree[Token]:
    """
    Lark's tree for the pre-lexed code, or its failure placed in the
    source.
    """
    try:
        return _from_tokens(code_tokens(lexed.code))
    except UnexpectedToken as failure:
        comment = isinstance(failure.token, Token) and
            failure.token.type in{
            "NOTE",
            "DOC",
        }
        message = (
            "a comment cannot stand inside an enclosure"
            if comment
            else "unexpected input"
        )
        raise FplError(
            placed(source, lexed, failure.pos_in_stream), message
        ) from None
    except UnexpectedInput as failure:
        raise FplError(
            placed(source, lexed, failure.pos_in_stream), "unexpected input"
        ) from None
    except MisindentedError as failure:
        where_ = placed(source, lexed, failure.token.end_pos)
```

```
        raise FplError(where_, "dedent to a level never opened") from None
```

A failure points into the source, not into the code string.

```
def within(source: str, line: int, col: int) -> bool:
    """
    Line and column name a character of the source, or the position just
    past a line.
    """
    lines = source.split("\n")
    return 1 <= line <= len(lines) and 1 <= col <= len(lines[line - 1]) + 1


def reported(source: str, line: int, column: int) -> bool:
    """
    What Lark reports for a failure: a position within the source, or
    line -1 for its end.
    """
    return line == -1 or within(source, line, column)


def inside(result: Span, source: str) -> bool:
    """
    The position given is within the source.
    """
    return within(source, result.line, result.col)


@icontract.require(reported)
@icontract.ensure(inside)
def where(source: str, line: int, column: int) -> Span:
    """
    The position of a parse failure. Lark gives -1 for the end of the
    input; that is the position just after the last character.
    """
    if line >= 1:
        return Span(line, column)
    lines = source.split("\n")
    return Span(len(lines), len(lines[-1]) + 1)


def placed(source: str, lexed: Prelexed, offset: int | None) -> Span:
    """
```

```
    The source position of a code offset; Lark's unknown or end position
    is the source's end.
    """
    if offset is None or offset < 0:
        return where(source, -1, -1)
    at = Lines(source).span(lexed.origin[offset])
    return where(source, at.line, at.col)
```

One walk of the Lark tree, into frames and cells.

```
class _Build:
    """
    The surface AST from Lark's tree, every node placed through the
    code's origins.
    """

    def __init__(self, source: str, lines: Lines, lexed: Prelexed) -> None:
        self.source, self.lines, self.lexed = source, lines, lexed

    def at(self, tree: Tree[Token], outer: Span) -> Span:
        """
        Where a subtree starts, or `outer` for one that matched nothing.
        """
        if tree.meta.empty:
            return outer
        return self.lines.span(self.lexed.origin[tree.meta.start_pos])

    def program(self, tree: Tree[Token], outer: Span) -> Program:
        """
        The start rule: the program's lines.
        """
        return Program(self.subtrees(tree, outer, self.line), outer)

    def subtrees[T](
        self,
        tree: Tree[Token],
        outer: Span,
        build: Callable[[Tree[Token], Span], T],
    ) -> tuple[T, ...]:
        """
        Each child subtree built.
        """
        return tuple(
```

```
        build(child, outer)
        for child in tree.children
        if isinstance(child, Tree)
    )

def line(self, tree: Tree[Token], outer: Span) -> Line:
    """
    frames (NOTE | DOC)? _NL block?; a ;; line holds its comment and
    no frames.
    """
    span = self.at(tree, outer)
    frames, *rest = tree.children
    assert isinstance(frames, Tree)
    lines = tuple(
        line
        for child in rest
        if isinstance(child, Tree)
        for line in self.subtrees(child, span, self.line)
    )
    comment = next(
        (self.comment(c, frames) for c in rest if isinstance(c, Token)),
        None,
    )
    return Line(self.held(frames, span, comment), lines, span, comment)

def held(
    self, frames: Tree[Token], span: Span, comment: Comment | None
) -> tuple[Frame, ...]:
    """
    A line's frames; a ;; line holds none.
    """
    if comment is not None and comment.level > 1:
        return ()
    return self.subtrees(frames, span, self.frame)

def comment(self, token: Token, frames: Tree[Token]) -> Comment:
    """
    A note and its continuation lines, or a ;; comment, refused
    after code on its line.
    """
    span = self.lines.span(self.lexed.origin[token.start_pos or 0])
    level = (
        1 if token.type == "NOTE" else len(token) - len(token.lstrip(";"))
```

```
    )
    if level > 1 and _code(frames):
        raise FplError(
            span, f"a {';' * level} comment stands on a line of its own"
        )
    return Comment(
        level, tuple(part.lstrip("\t") for part in token.split("\n")), span
    )

def frames(self, tree: Tree[Token], outer: Span) -> tuple[Frame, ...]:
    """
    frame (_BAR frame)*
    """
    return self.subtrees(tree, outer, self.frame)

def frame(self, tree: Tree[Token], outer: Span) -> Frame:
    """
    _TABS? (cell (_TABS cell)* _TABS?)?; empty, it starts where its
    enclosing node does.
    """
    span = self.at(tree, outer)
    return Frame(self.subtrees(tree, span, self.cell), span)

def cell(self, tree: Tree[Token], outer: Span) -> Cell:
    """
    item+
    """
    span = self.at(tree, outer)
    return Cell(
        tuple(self.item(child, span) for child in tree.children), span
    )

def item(self, child: Tree[Token] | Token, outer: Span) -> Item:
    """
    A token, a string placeholder, or an enclosure.
    """
    if isinstance(child, Tree):
        span = self.at(child, outer)
        (frames,) = self.subtrees(child, span, self.frames)
        return Enclosure(PAIRS[str(child.data)], frames, span)
    start = child.start_pos or 0
    span = self.lines.span(self.lexed.origin[start])
    stashed = self.lexed.stash.get(start)
```

```
    if stashed is not None:
        return self.text(stashed, span)
    return self.word(str(child), span)

def word(self, token: str, span: Span) -> Word:
    """
    The affix pass on one token.
    """
    read_ = shape(token)
    if read_ is None:
        raise FplError(span, f"a modifier ends its word: {token}")
    prefix, kind, body, mods = read_
    return Word(prefix, kind, body, mods, span)

def text(self, stashed: Stashed, span: Span) -> Text:
    """
    A string's interior: a raw one whole; an interpolating one split
    into text, with the incidental indentation of its continuation
    lines dropped (S26), and    islands read.
    """
    if stashed.kind == "raw":
        return Text(
            "raw", (self.source[stashed.start : stashed.end],), span
        )
    parts: list[str | Program] = []
    at = stashed.start
    while at < stashed.end:
        island = self.source.find(" ", at, stashed.end)
        upto = stashed.end if island < 0 else island
        if upto > at:
            parts.append(
                self.source[at:upto].replace(
                    "\n" + "\t" * stashed.depth, "\n"
                )
            )
        at = (
            upto if island < 0 else self.island(island, stashed.end, parts)
        )
    return Text("str", tuple(parts), span)

def island(self, at: int, end: int, parts: list[str | Program]) -> int:
    """
    Read the    island opening at `at` into parts; return the
```

```
        offset after it.
        """
        after = counted(self.source, at, BRACKETS[" "], end)
        if after is None:
            raise FplError(self.lines.span(at), "  never closed")
        parts.append(read(self.source, self.lines, at + 1, after - 1))
        return after
```

`_code` says whether a line holds code. `_parse` runs the replay, the indenter and this walk, and the name `parse` is that pass.

```
def _code(frames: Tree[Token]) -> bool:
    """
    A line's frames hold code: a bar, or a frame with cells.
    """
    return len(frames.children) > 1 or any(
        isinstance(frame, Tree) and frame.children for frame in frames.children
    )


def _parse(tokens: Tokens) -> Program:
    """
    The parser Lark over the lexer Lark's tokens.
    """
    source, lexed = tokens.source, tokens.lexed
    try:
        tree = _from_tokens(_as_tokens(tokens.raw))
    except UnexpectedToken as failure:
        comment = isinstance(failure.token, Token) and failure.token.type in {
            "NOTE",
            "DOC",
        }
        message = (
            "a comment cannot stand inside an enclosure"
            if comment
            else "unexpected input"
        )
        raise FplError(
            placed(source, lexed, failure.pos_in_stream), message
        ) from None
    except UnexpectedInput as failure:
        raise FplError(
            placed(source, lexed, failure.pos_in_stream), "unexpected input"
```

```
        ) from None
    except MisindentedError as failure:
        where_ = placed(source, lexed, failure.token.end_pos)
        raise FplError(where_, "dedent to a level never opened") from None
    lines = Lines(source)
    return _Build(source, lines, lexed).program(tree, lines.span(0))


parse = Pass("parse", _parse)
```

The cell below runs a program through that pass. Change the source and run it again.

```
import fpl

fpl.run.default.src("""
1 | 1 2 3 +
""")
```

01.02.03 Resolve

01.02.03 Resolve

`Syntax → Resolved`

01.02.02-parse is the previous arrow. 01.02.04-expand is the next.

This page is the signature. The walker does not have this pass on its own yet.

01.02.04 Expand

01.02.04 Expand

`Resolved → Expanded`

01.02.03-resolve is the previous arrow. 01.02.06-elaborate is the next.

This page is the signature. The walker does not have this pass on its own yet.

01.02.05 Desugar

01.02.05 Desugar

Today this is `Syntax → Core`. 01.02.03-resolve, 01.02.04-expand, and 01.02.07-lower are the arrows it stands in for.

```
"""
Surface AST to core AST, and core back to surface for the printer. All
sugar is erased here.

A line runs on a fresh stack. Its frames join into one postfix sequence:
the bar carries the running arity balance across, and a frame whose
words would reach below the balance it starts on is a section, pushed
whole as a quotation (decision f). Literals side by side in a cell
strand. The lines of a block fill the inputs of their head, before its
tokens: a child under a value slot is its code, run at once; under a
thunk or code slot, a quotation (S49 rule 5). The block of a `:` head is
its body, whose balance starts at the effect line's inputs and runs on
from line to line. ( ) puts its head after its arguments. Quotation
bodies are code, never sections. →x names the top for the rest of its
line, body or quotation, where x pushes it (decision f); #x is a symbol;
{ } pairs literal keys with one item each. A head `name/` mounts its
block as a directory: its definitions are named by path, and each
directory keeps an ordered log of its definitions, subdirectories and
`#name bind` mounts, where a later entry shadows an earlier one. A
word's body is its own directory (decision f), so a name in it is looked
up from there outward, `..` from the directory holding the word, and
every word w has w/history, w/doc, its docstring, and w/effect. A
`match` line in a body takes the values its balance counts, and each
line of its block is a row: one pattern cell per value, then at most a
body cell. A pattern is _, a name it binds, a literal, $x, ( constructor
patterns ) or p  test; a match with no row of only _ and names makes
its word's effect +fail. A line with no code, a comment's or the empty
first line, is no statement and no child; its comment is not carried
into the core, but for a word's docs (fpl/trivia.py). Anything outside
the implemented set is refused before evaluation (hole
unimplemented-words), at the source's start; but a call nothing answers
is refused at its word, as unknown or as not run yet (PLANNED). A word
```

```
with two or more keys, or one with a typed input, is dispatched (design
09 §1.1): its i-th key of arity n is the clause f/n/i, and f/n its
dispatcher, one row per clause guarding each typed input with its type
word and calling the clause, the most typed inputs first and the newest
first among as many; a call takes the arity group its balance picks.
"""

from collections import ChainMap
from collections.abc import Callable, Iterable, Iterator
from dataclasses import dataclass, field, replace
from decimal import Decimal
from functools import partial
from itertools import combinations, groupby
from typing import NoReturn, assert_never

from fpl import trivia
from fpl.arrow import Pass
from fpl.ast_core import (
    DIGITS,
    EFFECTS,
    PLANNED,
    Atom,
    Bind,
    Call,
    Define,
    Dict,
    Effect,
    Equal,
    Guarded,
    Inverse,
    Keyed,
    Listed,
    Match,
    Node,
    Number,
    Pattern,
    Push,
    Quotation,
    Refuse,
    Row,
    Run,
    Slot,
    Statement,
```

```
    Strand,
    Symbol,
    Value,
    Var,
    Wild,
)
from fpl.ast_surface import (
    Cell,
    Comment,
    Enclosure,
    Frame,
    Item,
    Line,
    Program,
    Text,
    Word,
)
from fpl.errors import FplError, Span, UnresolvedError
from fpl.print import render

START = Span(1, 1)
ARROWS = ("→", "->")
LOCAL = Effect((), ("x",))
HISTORY = Effect((), ("h",))
DOC = Effect((), ("d",))
QUERIES = {"history": HISTORY, "doc": DOC, "effect": Effect((), ("e",))}
PRIME = "\N{PRIME}"
TYPES = frozenset(("Int", "Decimal", "Text", "Symbol"))

type Here = tuple[str, ...]
type Part = str | None
type Key = tuple[Part, ...]
type Head = tuple[Effect, Span]
type Pair = tuple[int, int]
type Clause = tuple[str, Effect]


def unimplemented() -> NoReturn:
    """
    Refuse what no part implements yet, at the source's start.
    """
    raise FplError(START, "no evaluator yet")
```

```
def missing(word: Word, directory: bool) -> UnresolvedError:
    """
    The refusal of a call nothing answers, at the word and naming it as
    written: `unknown word: w`; but `no evaluator yet: w` where w names
    a directory, which is not called, or is a word the language's
    examples call (PLANNED).
    """
    name = "/".join(word.body)
    reason = (
        "no evaluator yet" if directory or name in PLANNED else "unknown word"
    )
    return UnresolvedError(word.span, f"{reason}: {word.prefix}{name}")


def desugar(program: Program) -> tuple[Statement, ...]:
    """
    The program's lines as statements, in order, a directory's
    definitions named by path. Every effect is known before any line is
    read, so a word may be used above its definition; a later definition
    shadows.
    """
    lines = coded(program.lines)
    catalog = Catalog({(): list(EFFECTS)}, dict(EFFECTS))
    catalog.enter(lines, ())
    catalog.settle()
    for here, log in catalog.logs.items():
        for entry in log:
            if isinstance(entry, Mount):
                catalog.directory(here, entry.name)
    return tuple(
        _Desugar(catalog, trivia.docstrings(program)).statements(lines, ())
    )


@dataclass(frozen=True)
class Mount:
    """
    #name bind: the directory name, as seen from the one it is bound in.
    """

    name: str
```

```
@dataclass
class Catalog:
    """
    Every directory's log, by path, and the effect of every word, by its
    path joined. A word is a directory too, holding its history. Each
    word's heads by key, in the order first written; each dispatched
    word's arities; the queries that answer at another path, and those
    refused; per arity group the pairs of clauses that cross typed two
    ways at one input.
    """

    logs: dict[Here, list[str | Mount]]
    effects: dict[str, Effect]
    heads: dict[str, dict[Key, Head]] = field(
        default_factory=dict[str, dict[Key, Head]]
    )
    groups: dict[str, tuple[int, ...]] = field(
        default_factory=dict[str, tuple[int, ...]]
    )
    aliases: dict[str, str] = field(default_factory=dict[str, str])
    refused: dict[str, str] = field(default_factory=dict[str, str])
    crossings: dict[str, list[Pair]] = field(
        default_factory=dict[str, list[Pair]]
    )

    def enter(self, lines: tuple[Line, ...], here: Here) -> None:
        """
        Log the definitions, subdirectories and mounts of one
        directory's lines.
        """
        log = self.logs.setdefault(here, [])
        for line in lines:
            head, name = definition(line), directory(line)
            if head is not None:
                log.append(head[0])
                self.keyed("/".join((*here, head[0])), head[1], line.span)
                self.queried((*here, head[0]), head[1], [])
            elif name is not None:
                log.append(name)
                self.enter(coded(line.block), (*here, name))
            elif here:
                log.append(Mount(mount(line)))
```

```
def keyed(self, path: str, effect: Effect, span: Span) -> None:
    """
    Note a definition's head by its key, a same key shadowing in its
    place; it must take the slot kinds and leave the values of every
    other key of its arity, else it is refused at its head (design
    09 §2.3).
    """
    heads, key = self.heads.setdefault(path, {}), keyed(effect)
    group = [other for other in heads if len(other) == len(key)]
    at = group.index(key) + 1 if key in group else len(group) + 1
    for i, other in enumerate(group, 1):
        if other != key:
            agree(
                f"{path}/{len(key)}",
                (at, effect, span),
                (i, heads[other][0]),
            )
    heads[key] = (effect, span)

def queried(self, here: Here, effect: Effect, entries: list[str]) -> None:
    """
    A word at here, its effect and its queries, its log holding
    entries before them.
    """
    path = "/".join(here)
    self.effects[path] = effect
    self.effects |= {f"{path}/{query}": e for query, e in QUERIES.items()}
    self.logs.setdefault(here, []).extend([*entries, *QUERIES])

def settle(self) -> None:
    """
    Dispatch each word with two or more keys, or one with a typed
    input.
    """
    builtins = TYPES - {path.rsplit("/", 1)[-1] for path in self.heads}
    for path, heads in self.heads.items():
        if len(heads) > 1 or any(
            typings(effect) for effect, _ in heads.values()
        ):
            self.dispatched(path, builtins)

def clauses(self, path: str, n: int) -> list[Head]:
```

```
    """
    A word's clauses of arity n, by path ordinal.
    """
    return [
        head for key, head in self.heads[path].items() if len(key) == n
    ]

def ordinal(self, path: str, effect: Effect) -> int:
    """
    The ordinal of a clause's key among its arity's.
    """
    key = keyed(effect)
    return [k for k in self.heads[path] if len(k) == len(key)].index(
        key
    ) + 1

def dispatched(self, path: str, builtins: frozenset[str]) -> None:
    """
    A word's arity groups f/n, each with its clauses f/n/i, of which
    two whose typed inputs cross are refused unless builtins make
    them disjoint or their meet is a clause or is typed two ways at
    one input; f/n's doc and history are its clause's when it has
    one, refused naming its clauses otherwise; f's queries are those
    of its only group, refused naming its groups when it has more.
    """
    here = tuple(path.split("/"))
    groups = {
        len(key): self.clauses(path, len(key)) for key in self.heads[path]
    }
    self.groups[path] = tuple(sorted(groups))
    for n, clauses in groups.items():
        self.crossings[f"{path}/{n}"] = crossed(
            f"{path}/{n}", clauses, builtins
        )
        self.logs[here].append(str(n))
        ordinals = [str(i) for i in range(1, len(clauses) + 1)]
        self.queried(
            (*here, str(n)),
            shared([effect for effect, _ in clauses]),
            ordinals,
        )
        for i, (clause, _) in zip(ordinals, clauses, strict=True):
            self.queried((*here, str(n), i), clause, [])
```

```
            self.answered(
                f"{path}/{n}", [f"{path}/{n}/{i}" for i in ordinals], "clause"
            )
        self.effects[path] = self.effects[f"{path}/{self.groups[path][0]}"]
        self.answered(
            path, [f"{path}/{n}" for n in self.groups[path]], "group"
        )

    def answered(self, path: str, parts: list[str], kind: str) -> None:
        """
        path's doc and history, and at a word its effect, answer as at
        its one part; with more they are refused naming them.
        """
        queries = ("history", "doc") if kind == "clause" else tuple(QUERIES)
        for query in queries:
            asked = f"{path}/{query}"
            if len(parts) > 1:
                self.refused[asked] = (
                    f"{asked} names more than one {kind}: {' '.join(parts)}"
                )
                continue
            answer = f"{parts[0]}/{query}"
            self.aliases[asked] = self.aliases.get(answer, answer)
            if answer in self.refused:
                self.refused[asked] = self.refused[answer].replace(
                    answer, asked, 1
                )

    def dispatch(self, path: str, balance: int) -> str:
        """
        The arity group a call on balance values takes: the largest
        arity it saturates, else the smallest; the path itself for a
        word not dispatched.
        """
        groups = self.groups.get(path)
        if groups is None:
            return path
        fits = [n for n in groups if n <= balance]
        return f"{path}/{max(fits, default=groups[0])}"

    def directory(self, here: Here, name: str) -> Here:
        """
        The directory a name reaches from here outward; none is refused.
```

```
    """
    for depth in range(len(here), -1, -1):
        if (*here[:depth], name) in self.logs:
            return (*here[:depth], name)
    unimplemented()

def lookup(self, here: Here, segments: Here) -> str | None:
    """
    The word a path names in one directory: the latest entry that
    holds it wins.
    """
    for entry in reversed(self.logs.get(here, [])):
        found = self.through(here, entry, segments)
        if found is not None:
            return found
    return None

def through(
    self, here: Here, entry: str | Mount, segments: Here
) -> str | None:
    """
    The word a path names through one entry: a subdirectory looks on
    in its own log, a mount only at the words its directory defines,
    so mounts never chain.
    """
    if isinstance(entry, Mount):
        path = "/".join((*self.directory(here, entry.name), *segments))
        return path if path in self.effects else None
    if entry != segments[0]:
        return None
    if len(segments) > 1:
        return self.lookup((*here, entry), segments[1:])
    path = "/".join((*here, entry))
    return path if path in self.effects else None

def find(self, here: Here, segments: Here) -> str | None:
    """
    The word a path names from here, then each enclosing directory,
    or None.
    """
    for depth in range(len(here), -1, -1):
        found = self.lookup(here[:depth], segments)
        if found is not None:
```

```
                return found
        return None

    def listed(self, here: Here, segments: Here) -> bool:
        """
        The path names a directory from here, then from each enclosing
        directory, or in a directory mounted in one of them.
        """
        for depth in range(len(here), -1, -1):
            there = here[:depth]
            roots = [there] + [
                self.directory(there, entry.name)
                for entry in self.logs.get(there, [])
                if isinstance(entry, Mount)
            ]
            if any((*root, *segments) in self.logs for root in roots):
                return True
        return False

    def resolve(self, here: Here, segments: Here) -> str:
        """
        The word a path names from here, then each enclosing directory;
        none is refused.
        """
        found = self.find(here, segments)
        if found is None:
            unimplemented()
        return found


def directory(line: Line) -> str | None:
    """
    The name of a `name/` head; a head with no block, or a deeper path,
    is refused.
    """
    items = [
        item
        for frame in line.frames
        for cell in frame.cells
        for item in cell.items
    ]
    match items:
        case [Word(prefix="", kind="path", body=(name, ""), mods="")] if (
```

```
            line.block
        ):
            return name
        case [Word(kind="path", body=(*_, "")), *_]:
            unimplemented()
        case _:
            return None


def mount(line: Line) -> str:
    """
    The name of a `#name bind` line; any other line in a directory is
    refused.
    """
    items = [
        item
        for frame in line.frames
        for cell in frame.cells
        for item in cell.items
    ]
    match items:
        case [Word(prefix="#", kind="name", body=(name,), mods=""), word] if (
            plain(word) == "bind"
        ):
            return name
        case _:
            unimplemented()


def coded(lines: tuple[Line, ...]) -> tuple[Line, ...]:
    """
    The lines that hold code: a bar, or an item in a frame. A line with
    none is a comment's or the empty first line, and heading a block it
    is refused (holes comment-heads-block, empty-first-line).
    """
    kept: list[Line] = []
    for line in lines:
        if trivia.code(line):
            kept.append(line)
        elif line.block:
            unimplemented()
    return tuple(kept)
```

```
def definition(line: Line) -> tuple[str, Effect] | None:
    """
    The name and effect of a `name : ins -- outs` line; None for a line
    that is not one.
    """
    if not trivia.head(line):
        return None
    cells = line.frames[0].cells
    if len(line.frames) + len(cells) > 2:
        unimplemented()
    return effect_line(cells[0].items)


def effect_line(items: tuple[Item, ...]) -> tuple[str, Effect]:
    """
    name : ins -- outs, each a plain name or `name: Type` (S49); an
    output's type is read past.
    """
    name = plain(items[0])
    declared = typed(items[2:])
    names = [pair[0] for pair in declared]
    if name is None or "--" not in names:
        unimplemented()
    cut = names.index("--")
    slots: tuple[Slot, ...] = tuple(kind for _, kind, _ in declared[:cut])
    types = tuple(part for _, _, part in declared[:cut])
    return name, Effect(
        tuple(names[:cut]), tuple(names[cut + 1 :]), slots=slots, types=types
    )


def typed(items: tuple[Item, ...]) -> list[tuple[str, Slot, Part]]:
    """
    Each plain name with its slot and type: `name: Type` has the slot
    its type makes, a bare name is an untyped value (hole
    bare-slot-names); ` ` is refused.
    """
    declared: list[tuple[str, Slot, Part]] = []
    rest = iter(items)
    for item in rest:
        name = plain(item)
        if isinstance(item, Enclosure) and item.pair == "prefix":
```

```
            declared.append(("_", "value", spelt(item)))
        elif name is None or name == " ":
            unimplemented()
        elif ascribed(item):
            declared.append((name[:-1], *slot(next(rest, None))))
        else:
            declared.append((name, "value", None))
    return declared


def slot(kind: Item | None) -> tuple[Slot, Part]:
    """
    The slot a type makes (S49 rule 5), and its type: [ ] a thunk, Code
    code, both untyped; a name a value of that type (a word x -- b). A
    `name:` with no type after it is refused, and so is a compound type
    (hole compound-type).
    """
    if kind is None or plain(kind) == "--":
        unimplemented()
    if isinstance(kind, Enclosure) and kind.pair == "quotation":
        return "thunk", None
    part = plain(kind)
    if part is None:
        unimplemented()
    return ("code", None) if part == "Code" else ("value", part)


def ascribed(item: Item) -> bool:
    """
    A plain `name:`, whose type follows it.
    """
    name = plain(item)
    return name is not None and len(name) > 1 and name.endswith(":")


def grouped(part: Part) -> bool:
    """
    A slot's type is a group `( p )`, written as the printer writes it.
    """
    return part is not None and part.startswith("(")


def spelt(item: Item) -> str:
```

```
    """
    An item as the printer writes it.
    """
    line = Line((framed((item,)),), (), START)
    return render(Program((Line((), (), START), line), START)).strip()


def pins(items: list[Item]) -> bool:
    """
    Items holding a `$` pin, at any depth.
    """
    return any(
        (isinstance(item, Word) and item.prefix == "$")
        or (isinstance(item, Enclosure) and pins(contents(item)))
        for item in items
    )


def keyed(effect: Effect) -> Key:
    """
    A definition's key: per input its type, a thunk or code slot its
    kind (hole clause-identity).
    """
    pairs = zip(effect.slots, effect.types, strict=True)
    return tuple(part if kind == "value" else kind for kind, part in pairs)


def agree(
    group: str, later: tuple[int, Effect, Span], earlier: tuple[int, Effect]
) -> None:
    """
    Two clauses of one arity group take the same slot kinds and leave as
    many values, else the later is refused at its head.
    """
    at, effect, span = later
    i, other = earlier
    for position, (kind, before) in enumerate(
        zip(effect.slots, other.slots, strict=True), 1
    ):
        if kind != before:
            message = (
                f"{group}/{at} takes a {kind} at {position}, "
                f"{group}/{i} a {before}"
```

```
            )
            raise FplError(span, message)
    if len(effect.outs) != len(other.outs):
        message = (
            f"{group}/{at} leaves {len(effect.outs)}, "
            f"{group}/{i} {len(other.outs)}"
        )
        raise FplError(span, message)


def typings(effect: Effect) -> frozenset[int]:
    """
    The typed inputs of a clause, by position.
    """
    return frozenset(
        i for i, part in enumerate(effect.types) if part is not None
    )


def crossed(
    group: str, clauses: list[Head], builtins: frozenset[str]
) -> list[Pair]:
    """
    Refuse two clauses each typed where the other is not, unless
    disjoint, two different builtins at one position, or their meet,
    each input typed as either types it, is itself a clause: at the
    later head, naming the meet (design 09 §3.3). The pairs, by ordinal,
    whose meet one input would type two ways: only a run can tell
    whether both fit (§3.4).
    """
    written = {effect.types for effect, _ in clauses}
    undecided: list[Pair] = []
    for a, b in combinations(enumerate(clauses, 1), 2):
        first, second = a[1][0], b[1][0]
        ordered = typings(first) <= typings(second) or typings(
            second
        ) <= typings(first)
        if ordered or disjoint(first, second, builtins):
            continue
        meet = met(first, second)
        if meet is None:
            undecided.append((a[0], b[0]))
        elif meet.types not in written:
```

```
            (later, (_, span)), (earlier, _) = sorted(
                (a, b), key=lambda c: -c[1][1].line
            )
            message = (
                f"{group}/{later} is ambiguous with {group}/{earlier} "
                f"at {spoken(meet)}"
            )
            raise FplError(span, message)
    return undecided


def disjoint(a: Effect, b: Effect, builtins: frozenset[str]) -> bool:
    """
    Two clauses no value fits both of: two different builtins at one
    position.
    """
    return any(
        x != y and {x, y} <= builtins
        for x, y in zip(a.types, b.types, strict=True)
    )


def met(a: Effect, b: Effect) -> Effect | None:
    """
    Two clauses' meet: a's effect, each input typed as either types it;
    None when both type one input differently.
    """
    pairs = list(zip(a.types, b.types, strict=True))
    if any(x is not None and y is not None and x != y for x, y in pairs):
        return None
    return replace(a, types=tuple(x or y for x, y in pairs))


def spoken(effect: Effect) -> str:
    """
    An effect's inputs as a head writes them, `x: T` where typed, two
    spaces apart.
    """
    pairs = zip(effect.ins, effect.types, strict=True)
    return "  ".join(
        name if part is None else f"{name}: {part}" for name, part in pairs
    )
```

```
def shared(clauses: list[Effect]) -> Effect:
    """
    A dispatcher's effect: its first clause's, keeping a type only where
    all agree.
    """
    first = clauses[0]
    types = tuple(
        part if all(c.types[i] == part for c in clauses) else None
        for i, part in enumerate(first.types)
    )
    return replace(first, types=types)


def plain(item: Item) -> str | None:
    """
    A name with no sigil or modifier, or None.
    """
    if (
        isinstance(item, Word)
        and item.kind == "name"
        and not (item.prefix or item.mods)
    ):
        return item.body[0]
    return None


def atom(item: Item) -> Atom:
    """
    The value of a literal.
    """
    return text(item) if isinstance(item, Text) else number(item)


def text(item: Text) -> str:
    """
    A string without islands.
    """
    texts = [part for part in item.parts if isinstance(part, str)]
    if len(texts) < len(item.parts):
        unimplemented()
    return "".join(texts)
```

```
def number(item: Item) -> Number:
    """
    An integer or a decimal, kept exact as written; ϖ,  and a sigil or
    modifier are refused, and so is a numeral of more than DIGITS
    digits, at its position.
    """
    if (
        not isinstance(item, Word)
        or item.prefix
        or item.mods
        or not item.body[0][-1].isdigit()
    ):
        unimplemented()
    numeral = item.body[0]
    if sum(map(str.isdecimal, numeral)) > DIGITS:
        raise FplError(item.span, "number too long")
    return Decimal(numeral) if "." in numeral else int(numeral)


def bare(word: Word) -> str:
    """
    The name after a sigil; a path or a modifier is refused.
    """
    if word.kind != "name" or word.mods:
        unimplemented()
    return word.body[0]


def contents(enclosure: Enclosure) -> list[Item]:
    """
    The items of an enclosure, bars and tabs aside.
    """
    return [
        item
        for frame in enclosure.frames
        for cell in frame.cells
        for item in cell.items
    ]


def scoped(code: tuple[Node, ...]) -> tuple[Node, ...]:
    """
```

```
    Code with each binder holding the rest of it as its body.
    """
    out: list[Node] = []
    for node in reversed(code):
        if isinstance(node, Bind):
            out = [
                Bind(node.name, node.body + tuple(reversed(out)), node.span)
            ]
        else:
            out.append(node)
    return tuple(reversed(out))


def fallible(code: tuple[Node, ...]) -> bool:
    """
    Some match in the code, outside a quotation, has no row that catches
    every value.
    """
    for node in code:
        if isinstance(node, Match) and not any(map(catches, node.rows)):
            return True
        if isinstance(node, Bind) and fallible(node.body):
            return True
    return False


def catches(row: Row) -> bool:
    """
    Every pattern of the row is _ or a name.
    """
    return all(isinstance(pattern, Wild | Var) for pattern in row.patterns)


def matcher(line: Line) -> bool:
    """
    A line of the one word match, with the rows as its block.
    """
    items = [
        item
        for frame in line.frames
        for cell in frame.cells
        for item in cell.items
    ]
```

```
    return bool(line.block) and len(items) == 1 and plain(items[0]) == "match"


def literal(item: Item) -> bool:
    """
    A number or a string: what strands.
    """
    return isinstance(item, Text) or (
        isinstance(item, Word) and item.kind == "number"
    )


class _Desugar:
    """
    Lines to code, with the effect of every word known.
    """

    def __init__(self, catalog: Catalog, docs: dict[int, str]) -> None:
        """
        Keep the catalog, the docstrings by the line they document, and
        the effect of every word a line may call: the builtins' and the
        definitions', by path.
        """
        self.catalog = catalog
        self.docs = docs
        self.effects = ChainMap(catalog.effects)
        self.here: Here = ()
        self.dispatchers: set[str] = set()

    def statements(
        self, lines: tuple[Line, ...], here: Here
    ) -> Iterator[Statement]:
        """
        A directory's definitions and, at the top, the lines to run.
        """
        for line in lines:
            head, name = definition(line), directory(line)
            if head is not None:
                path = (*here, head[0])
                code = self.code(line, head[1], path)
                effect = replace(head[1], fails=fallible(code))
                doc = self.docs.get(line.span.line, "")
                yield from self.defined(
```

```
                    Define("/".join(path), effect, code, doc, line.span), here
                )
            elif name is not None:
                yield from self.statements(coded(line.block), (*here, name))
            elif not here:
                yield Run(self.body((line,), 0, here), line.span)

    def code(self, line: Line, effect: Effect, here: Here) -> tuple[Node, ...]:
        """
        A definition's code: its body; with a head group, one row
        binding a fresh name at every other input and the group's
        pattern at each group, the other inputs pushed back in order,
        then the body, which sees the group's names (design 09 §2.4). A
        `$` pin in a head group is refused (hole head-group-pin).
        """
        groups = iter(
            i
            for i in line.frames[0].cells[0].items[2:]
            if isinstance(i, Enclosure) and i.pair == "prefix"
        )
        if not any(map(grouped, effect.types)):
            return self.body(line.block, len(effect.ins), here)
        outer, patterns, repush = self.effects, list[Pattern](), list[Node]()
        self.here = here
        for name, part in zip(fresh(effect), effect.types, strict=True):
            if grouped(part):
                group = next(groups)
                if pins(contents(group)):
                    unimplemented()
                patterns.append(self.simple(group))
            else:
                patterns.append(Var(name))
                repush.append(Call(name, line.span))
        body = self.body(line.block, len(repush), here)
        self.effects = outer
        return (Match((Row(tuple(patterns), (*repush, *body)),), line.span),)

    def defined(self, define: Define, here: Here) -> Iterator[Define]:
        """
        A definition; of a dispatched word, its clause, and after the
        first clause of an arity its dispatcher at that clause's head
        (design 09 §2.4).
        """
```

```
    if define.name not in self.catalog.groups:
        yield define
        return
    n = len(define.effect.ins)
    clause = replace(
        define,
        clause=(n, self.catalog.ordinal(define.name, define.effect)),
    )
    yield clause
    yield from tests(clause)
    group = f"{define.name}/{n}"
    if group not in self.dispatchers:
        self.dispatchers.add(group)
        yield self.dispatcher(define.name, n, clause.span, here)

def dispatcher(self, name: str, n: int, span: Span, here: Here) -> Define:
    """
    The dispatcher of an arity group: a row per clause and a
    refusing row per pair that crosses typed two ways at one input,
    the most typed inputs first; among as many the clauses' rows,
    newest first, then the refusing ones (design 09 §3.2); +fail
    when no row catches every value.
    """
    group = f"{name}/{n}"
    clauses = [
        (f"{group}/{i}", c)
        for i, (c, _) in enumerate(self.catalog.clauses(name, n), 1)
    ]
    ranked = [
        (
            (-len(typings(effect)), 0, -i, 0),
            self.guarding(word, effect, span, here),
        )
        for i, (word, effect) in enumerate(clauses, 1)
    ]
    for i, j in self.catalog.crossings[group]:
        sides = (clauses[i - 1], clauses[j - 1])
        count = len(typings(sides[0][1]) | typings(sides[1][1]))
        ranked.append(
            ((-count, 1, i, j), self.refusing(name, sides, span, here))
        )
    rows = tuple(
        row for _, row in sorted(ranked, key=lambda rank: rank[0])
```

```
    )
    code = (Match(rows, span),)
    effect = replace(self.catalog.effects[group], fails=fallible(code))
    words = tuple(word for word, _ in clauses)
    return Define(
        name, effect, code, span=span, clause=(n,), clauses=words
    )

def guarding(
    self, word: str, effect: Effect, span: Span, here: Here
) -> Row:
    """
    A clause's dispatcher row: at each input a fresh name, its
    slot's name, a prime and its position, guarding each typed input
    with its test, then every input pushed back in order and the
    clause called.
    """
    names = fresh(effect)
    tests = zip(names, self.checks(word, effect, here), strict=True)
    patterns = tuple(
        guard(Var(name), (test,), span) for name, test in tests
    )
    return Row(
        patterns, (*(Call(name, span) for name in names), Call(word, span))
    )

def refusing(
    self, name: str, sides: tuple[Clause, Clause], span: Span, here: Here
) -> Row:
    """
    The row fitting where both clauses fit, each input under the
    first clause's test and then the second's, refusing the call as
    ambiguous (design 09 §2.4).
    """
    tests = [self.checks(word, effect, here) for word, effect in sides]
    patterns = tuple(
        guard(Wild(), column, span) for column in zip(*tests, strict=True)
    )
    both = " and ".join(word for word, _ in sides)
    return Row(
        patterns,
        (Refuse(f"ambiguous call to {name}: {both} both fit", span),),
    )
```

```
def checks(
    self, word: str, effect: Effect, here: Here
) -> list[str | None]:
    """
    The test of each input of a clause, None where it is untyped.
    """
    return [
        None if part is None else self.tested(word, part, i, here)
        for i, part in enumerate(effect.types, 1)
    ]

def tested(self, word: str, part: str, position: int, here: Here) -> str:
    """
    The word testing a clause's typed input: a group's generated
    word, else the word its type names from the directory of its
    definition, on one value.
    """
    if grouped(part):
        return f"{word}{PRIME}{position}"
    return self.catalog.dispatch(self.catalog.resolve(here, (part,)), 1)

def routed(
    self, nodes: tuple[Node, ...], balance: int
) -> tuple[Node, ...]:
    """
    The nodes, each call of a dispatched word taking the arity group
    its balance picks; a word reaching below the balance leaves it
    at what it adds.
    """
    out: list[Node] = []
    for node in nodes:
        taken = node
        if isinstance(node, Call) and not self.local(node.name):
            taken = Call(
                self.catalog.dispatch(node.name, balance), node.span
            )
        takes, leaves = self.arity(taken)
        balance = max(0, balance - takes) + leaves
        out.append(taken)
    return tuple(out)

def body(
```

```
    self, lines: tuple[Line, ...], balance: int, here: Here
) -> tuple[Node, ...]:
    """
    Lines on one stack in turn, each starting on the balance the one
    before left, their names looked up from here. A name bound in
    them dies with them, and one bound in a section with the
    section.
    """
    outer = self.effects
    self.here = here
    code: list[Node] = []
    for line in coded(lines):
        if matcher(line):
            code.append(self.match(line, balance))
            balance = 0
            continue
        parts = (
            *(
                partial(self.child, child, kind)
                for child, kind in self.filled(line)
            ),
            *(partial(self.frame, frame) for frame in line.frames),
        )
        for part in parts:
            before = self.effects
            written = part()
            nodes = self.routed(written, balance)
            after = self.balance(nodes, balance)
            if after is None:
                nodes, after = (
                    (Push(Quotation(scoped(self.routed(written, 0)))),),
                    balance + 1,
                )
                self.effects = before
            code.extend(nodes)
            balance = after
    self.effects = outer
    return scoped(tuple(code))

def balance(self, nodes: tuple[Node, ...], balance: int) -> int | None:
    """
    The balance after the nodes run on `balance` values; None if one
    reaches below.
```

```
    """
    for node in nodes:
        takes, leaves = self.arity(node)
        if balance < takes:
            return None
        balance += leaves - takes
    return balance

def arity(self, node: Node) -> tuple[int, int]:
    """
    How many values a node takes and leaves; a word's from its
    effect.
    """
    match node:
        case Push() | Keyed():
            return 0, 1
        case Bind():
            return 1, 0
        case Match():  # pragma: no cover -- never in a frame
            return len(node.rows[0].patterns), 0
        case Refuse():  # pragma: no cover -- generated, never routed
            return 0, 0
        case Call():
            effect = self.effects[node.name]
            return len(effect.ins), len(effect.outs)
        case _:
            assert_never(node)

def match(self, line: Line, arity: int) -> Match:
    """
    The rows of a match taking arity values; a block of comments
    alone, no rows, is refused at the word.
    """
    word = line.frames[0].cells[0].items[0]
    rows = tuple(self.row(row, arity) for row in coded(line.block))
    if not rows:
        raise FplError(word.span, "a match has no rows")
    return Match(rows, word.span)

def row(self, line: Line, arity: int) -> Row:
    """
    arity pattern cells, then at most a body cell, where the names
    they bind are read.
```

```
    """
    cells = [cell for frame in line.frames for cell in frame.cells]
    if (
        line.block
        or len(line.frames) > 1
        or len(cells) - arity not in (0, 1)
    ):
        raise FplError(line.span, f"a row is {arity} patterns and a body")
    outer = self.effects
    patterns = tuple(self.pattern(cell.items) for cell in cells[:arity])
    body = self.routed(
        self.frame(Frame(tuple(cells[arity:]), line.span)), 0
    )
    self.effects = outer
    return Row(patterns, scoped(body))

def pattern(self, items: tuple[Item, ...]) -> Pattern:
    """
    The one pattern of a cell or of ( p   test ).
    """
    found = self.patterns(items)
    if len(found) != 1:
        unimplemented()
    return found[0]

def patterns(self, items: tuple[Item, ...]) -> tuple[Pattern, ...]:
    """
    Items as patterns in turn, ` test` guarding the one before it.
    """
    found: list[Pattern] = []
    marks = iter(items)
    for item in marks:
        if plain(item) == " " and found:
            found[-1] = self.guarded(
                found[-1], next(marks, None), item.span
            )
        elif isinstance(item, Word) and ascribed(item):
            name = replace(item, body=(item.body[0][:-1],))
            found.append(
                self.guarded(
                    self.simple(name), next(marks, None), item.span
                )
            )
```

```
        else:
            found.append(self.simple(item))
    return tuple(found)

def guarded(
    self, pattern: Pattern, test: Item | None, span: Span
) -> Guarded:
    """
    A pattern that fits where it matches and its test word leaves 1.
    """
    if not isinstance(test, Word):
        unimplemented()
    return Guarded(
        pattern, self.catalog.dispatch(self.call(test).name, 1), span
    )

def simple(self, item: Item) -> Pattern:
    """
    _, $x of a bound name, a name it binds, ( constructor patterns
    ), ( p   test ) or a literal.
    """
    match item:
        case Word(prefix="", kind="name", body=("_",), mods=""):
            return Wild()
        case Word(prefix="$", kind="name", body=(name,), mods="") if (
            self.local(name)
        ):
            return Equal(Call(name, item.span))
        case Word() if plain(item) is not None:
            self.effects = self.effects.new_child({item.body[0]: LOCAL})
            return Var(item.body[0])
        case Enclosure(pair="prefix"):
            return self.inverse(contents(item))
        case _:
            return self.constant(item)

def local(self, name: str) -> bool:
    """
    A name bound by a binder or an earlier pattern.
    """
    return any(name in scope for scope in self.effects.maps[:-1])

def constant(self, item: Item) -> Equal:
```

```
    """
    A literal, matching by equality.
    """
    nodes = self.item(item)
    if len(nodes) != 1 or not isinstance(nodes[0], Push):
        unimplemented()
    return Equal(nodes[0])

def inverse(self, items: list[Item]) -> Pattern:
    """
    ( p   test ), or a constructor taking one pattern per input and
    leaving one value.
    """
    if not items:
        unimplemented()
    if len(items) > 1 and (plain(items[1]) == " " or ascribed(items[0])):
        return self.pattern(tuple(items))
    head, *args = items
    if not isinstance(head, Word):
        unimplemented()
    called = self.call(head).name
    found = self.patterns(tuple(args))
    name = self.catalog.dispatch(called, len(found))
    effect = self.effects[name]
    if len(effect.outs) != 1:
        raise FplError(head.span, f"{name} is not invertible")
    if len(found) != len(effect.ins):
        raise FplError(
            head.span, f"{name} takes {len(effect.ins)} patterns"
        )
    return Inverse(name, found, head.span)

def quote(self, build: Callable[[], tuple[Node, ...]]) -> Push:
    """
    Code built in a scope of its own, pushed as a quotation.
    """
    outer = self.effects
    code = self.routed(build(), 0)
    self.effects = outer
    return Push(Quotation(scoped(code)))

def children(self, line: Line) -> tuple[tuple[Node, ...], ...]:
    """
```

```
    Each line of the block in order, filling the head's inputs, the
    last child the top one: under a value slot its code, run at
    once; under a thunk or code slot, or past the head's inputs, a
    quotation (hole child-slots).
    """
    return tuple(
        self.child(child, kind) for child, kind in self.filled(line)
    )

def filled(self, line: Line) -> tuple[tuple[Line, Slot | None], ...]:
    """
    Each child with the slot it fills, None past the head's inputs;
    the head is read in a scope thrown away, so its binders reach
    neither the children nor its own reading.
    """
    block = coded(line.block)
    outer = self.effects
    inputs = self.inputs(self.routed(self.frames(line.frames), len(block)))
    self.effects = outer
    kinds: list[Slot | None] = [None] * len(block)
    kinds += inputs
    return tuple(zip(block, kinds[len(inputs) :], strict=True))

def child(self, line: Line, kind: Slot | None) -> tuple[Node, ...]:
    """
    A child's code, or its quotation, in a scope of its own, when it
    fills no value slot.
    """
    if kind == "value":
        return self.later(line)
    return (self.quote(partial(self.later, line)),)

def inputs(self, code: tuple[Node, ...]) -> tuple[Slot, ...]:
    """
    The slots code takes from below it, deepest first: the head
    phrase as a whole, so the inputs a word's code before it does
    not supply lie under those already taken (hole multi-word-head).
    A binder's input is a value.
    """
    taken: tuple[Slot, ...] = ()
    held = 0
    for node in code:
        takes, leaves = self.arity(node)
```

```
        effect = (
            self.effects[node.name]
            if isinstance(node, Call)
            else Effect(("x",) * takes, ())
        )
        short = max(0, takes - held)
        taken = effect.slots[:short] + taken
        held += short + leaves - takes
    return taken

def later(self, line: Line) -> tuple[Node, ...]:
    """
    A line as code to run later: its children, then its frames
    joined.
    """
    children = (node for part in self.children(line) for node in part)
    return (*children, *self.frames(line.frames))

def frames(self, frames: tuple[Frame, ...]) -> tuple[Node, ...]:
    """
    Frames joined, with no saturation.
    """
    return tuple(node for frame in frames for node in self.frame(frame))

def frame(self, frame: Frame) -> tuple[Node, ...]:
    """
    Cells in turn; within a cell, literals side by side strand.
    """
    code: list[Node] = []
    for cell in frame.cells:
        for strands, run in groupby(cell.items, key=literal):
            items = tuple(run)
            if strands and len(items) > 1:
                code.append(Push(Strand(tuple(map(atom, items)))))
            else:
                code.extend(
                    node for item in items for node in self.item(item)
                )
    return tuple(code)

def item(self, item: Item) -> tuple[Node, ...]:
    """
    One item's code.
```

```
    """
    match item:
        case Word():
            return (self.word(item),)
        case Text():
            return (Push(text(item)),)
        case Enclosure():
            return self.enclosure(item)
        case _:
            assert_never(item)

def word(self, word: Word) -> Node:
    """
    A number, a symbol, a binder or a call.
    """
    if word.kind == "number":
        return Push(number(word))
    if word.prefix == "#":
        return Push(Symbol(bare(word)))
    if word.prefix in ARROWS:
        name = (
            "/".join(word.body)
            if word.kind == "path" and word.body[-1]
            else bare(word)
        )
        self.effects = self.effects.new_child({name: LOCAL})
        return Bind(name, (), word.span)
    return self.call(word)

def call(self, word: Word) -> Call:
    """
    A word bound on its line, else the one its path names from here,
    or from the directory holding here after `..`.
    """
    path = "/".join(word.body)
    if not (word.prefix or word.mods) and any(
        path in s for s in self.effects.maps[:-1]
    ):
        return Call(path, word.span)
    here = self.origin(word)
    found = self.catalog.find(here, word.body)
    if found is None:
        raise missing(word, self.catalog.listed(here, word.body))
```

```
    if found in self.catalog.refused:
        raise FplError(word.span, self.catalog.refused[found])
    return Call(self.catalog.aliases.get(found, found), word.span)

def origin(self, word: Word) -> Here:
    """
    Where a word's lookup starts: here, or after `..` the directory
    holding here; a modifier or another sigil is refused.
    """
    match word.prefix:
        case "" if not word.mods:
            return self.here
        case "../" if self.here and not word.mods:
            return self.here[:-1]
        case _:
            unimplemented()

def enclosure(self, enclosure: Enclosure) -> tuple[Node, ...]:
    """
    [ ] a quotation, ( ) its head last,    a list of literals.
    """
    match enclosure.pair:
        case "quotation":
            return (self.quote(partial(self.frames, enclosure.frames)),)
        case "prefix":
            return self.prefix(enclosure)
        case "group":
            return (
                Push(
                    Listed(tuple(map(self.element, contents(enclosure))))
                ),
            )
        case "dict":
            return (self.dict(enclosure),)
        case _:
            assert_never(enclosure.pair)

def prefix(self, enclosure: Enclosure) -> tuple[Node, ...]:
    """
    ( head args ): each argument's code, then the head's; arguments
    never strand.
    """
    cells = [cell for frame in enclosure.frames for cell in frame.cells]
```

```
        if len(cells) != 1:
            unimplemented()
        head, *args = cells[0].items
        return tuple(
            node for item in (*args, head) for node in self.item(item)
        )

    def dict(self, enclosure: Enclosure) -> Keyed:
        """
        Keys and values in turn: a key a plain name, a value one item
        pushing one value; a key given twice is refused where it
        repeats.
        """
        items = contents(enclosure)
        if len(items) % 2:
            unimplemented()
        entries: dict[str, Node] = {}
        for key_item, value_item in zip(items[::2], items[1::2], strict=True):
            key, nodes = plain(key_item), self.routed(self.item(value_item), 0)
            if (
                key is None
                or len(nodes) != 1
                or self.arity(nodes[0]) != (0, 1)
            ):
                unimplemented()
            if key in entries:
                raise FplError(key_item.span, f"repeated key {key}")
            entries[key] = nodes[0]
        return Keyed(tuple(entries.items()), enclosure.span)

    def element(self, item: Item) -> Value:
        """
        A list element: an item that pushes one value.
        """
        nodes = self.item(item)
        if len(nodes) == 1 and isinstance(nodes[0], Push):
            return nodes[0].value
        unimplemented()


def resugar(statements: tuple[Statement, ...]) -> Program:
    """
    Core as source that desugars to the same code: no bar but between
```

```
    two literals, which would otherwise strand, no ( ), no block but a
    definition's one-line body and its docs; a dispatcher is generated,
    never written.
    """
    kept = (s for s in statements if not generated(s))
    return Program(
        tuple(line for s in kept for line in (*above(s), written(s))), START
    )


def generated(statement: Statement) -> bool:
    """
    A dispatcher, a group's test word or a clause with a group: made
    from a head, never written back (hole resugar-match).
    """
    if not isinstance(statement, Define):
        return False
    made = len(statement.clause) == 1 or PRIME in statement.name
    return made or any(map(grouped, statement.effect.types))


def guard(
    pattern: Pattern, tests: Iterable[str | None], span: Span
) -> Pattern:
    """
    The pattern guarded by each test in turn, the first innermost; None
    is no test.
    """
    for test in tests:
        if test is not None:
            pattern = Guarded(pattern, test, span)
    return pattern


def fresh(effect: Effect) -> list[str]:
    """
    A fresh name per input: its slot's name, a prime and its position
    (design 09 §2.4).
    """
    return [f"{name}{PRIME}{i}" for i, name in enumerate(effect.ins, 1)]


def tests(clause: Define) -> Iterator[Define]:
```

```
    """
    For each group of a clause, the word x -- b leaving 1 where the
    group matches, else 0, named by the clause's word, a prime and the
    group's position.
    """
    for position, part in enumerate(clause.effect.types, 1):
        match clause.code:
            case (Match(rows=(row,)),) if grouped(part):
                fits = Row((row.patterns[position - 1],), (Push(1),))
                code = Match((fits, Row((Wild(),), (Push(0),))), clause.span)
                name = f"{clause.word}{PRIME}{position}"
                yield Define(
                    name, Effect(("x",), ("b",)), (code,), span=clause.span
                )
            case _:
                pass


def listing(stacks: tuple[tuple[Value, ...], ...]) -> Program:
    """
    One line per stack, each a program that pushes that stack. A value
    nested deeper than the build can recurse to is refused at the
    source's start, as the parser refuses such source (issue #95).
    """
    try:
        return Program(
            tuple(
                Line(sugared(tuple(map(Push, s))), (), START) for s in stacks
            ),
            START,
        )
    except RecursionError:
        raise FplError(START, "nesting too deep to print") from None


def written(statement: Statement) -> Line:
    """
    One statement as a line.
    """
    match statement:
        case Run():
            return Line(sugared(statement.code), (), START)
        case Define():
```

```
            effect = statement.effect
            fails = ("+fail",) if effect.fails else ()
            ins = zip(effect.ins, effect.slots, effect.types, strict=True)
            taken = (
                item
                for name, kind, part in ins
                for item in declaration(name, kind, part)
            )
            head = (named(statement.name), named(":"), *taken, named("--"))
            body = (
                (Line(sugared(statement.code), (), START),)
                if statement.code
                else ()
            )
            outs = (*effect.outs, *fails)
            return Line(
                (framed((*head, *map(named, outs))),),
                documented(statement) + body,
                START,
            )
        case _:
            assert_never(statement)


def documented(statement: Define) -> tuple[Line, ...]:
    """
    A definition's docstring as the ;; lines opening its body; none when
    it has no body, as its docs are written above its head instead.
    """
    return doc_lines(statement.doc) if statement.code else ()


def above(statement: Statement) -> tuple[Line, ...]:
    """
    The ;; lines written above a statement: the docstring of a
    definition with no code, which under its head would be read as the
    next code line's (fpl/trivia.py); none else.
    """
    if isinstance(statement, Define) and not statement.code:
        return doc_lines(statement.doc)
    return ()
```

```
def doc_lines(doc: str) -> tuple[Line, ...]:
    """
    A docstring as ;; lines, one for each of its lines; none for the
    empty one.
    """
    texts = doc.split("\n") if doc else []
    return tuple(
        Line((), (), START, Comment(2, (f";; {text}",), START))
        for text in texts
    )


def sugared(code: tuple[Node, ...]) -> tuple[Frame, ...]:
    """
    Code as frames, a bar wherever a literal follows a literal.
    """
    frames: list[list[Item]] = [[]]
    before: Node | None = None
    for node in unfolded(code):
        if stranding(before) and stranding(node):
            frames.append([])
        frames[-1].extend(spelled(node))
        before = node
    return tuple(framed(tuple(items)) for items in frames)


def unfolded(code: tuple[Node, ...]) -> Iterator[Node]:
    """
    The nodes in the order written: a binder, then its body.
    """
    for node in code:
        yield node
        if isinstance(node, Bind):
            yield from unfolded(node.body)


def spelled(node: Node) -> tuple[Item, ...]:
    """
    The items that write one node; a binder's body is written after it.
    """
    match node:
        case Push():
            return shown(node.value)
```

```
        case Call():
            return (named(node.name),)
        case Bind():
            return (
                Word(
                    "→",
                    named(node.name).kind,
                    named(node.name).body,
                    "",
                    START,
                ),
            )
        case Keyed():
            pairs = tuple(
                item
                for key, n in node.entries
                for item in (named(key), *spelled(n))
            )
            return (Enclosure("dict", (framed(pairs),), START),)
        case Match() | Refuse():
            unimplemented()
        case _:
            assert_never(node)


def stranding(node: Node | None) -> bool:
    """
    A push of a number, a string or a strand.
    """
    return isinstance(node, Push) and isinstance(
        node.value, int | Decimal | str | Strand
    )


def framed(items: tuple[Item, ...]) -> Frame:
    """
    Items as a frame of one cell, or of none.
    """
    return Frame((Cell(items, START),) if items else (), START)


def declaration(name: str, kind: Slot, part: Part) -> tuple[Item, ...]:
    """
```

```
    An input as written: a value bare or `name: Type`, a thunk `name:
    []`, code `name: Code`. The type a thunk was declared with is not
    kept (hole thunk-type-dropped).
    """
    match kind:
        case "value" if part is not None:
            return (named(name + ":"), named(part))
        case "value":
            return (named(name),)
        case "thunk":
            return (
                named(name + ":"),
                Enclosure("quotation", (framed(()),), START),
            )
        case "code":
            return (named(name + ":"), named("Code"))
        case _:
            assert_never(kind)


def named(name: str) -> Word:
    """
    A name as written, a path when it holds a slash.
    """
    segments = tuple(name.split("/"))
    return Word(
        "", "path" if len(segments) > 1 else "name", segments, "", START
    )


def shown(value: Value) -> tuple[Item, ...]:
    """
    A value as the items that push it.
    """
    match value:
        case str():
            return (Text("str", (value,), START),)
        case Strand():
            return spread(value.items)
        case Listed() | Quotation() | Dict():
            return (enclosed(value),)
        case Symbol():
            return (Word("#", "name", (value.name,), "", START),)
```

```
        case _:
            numeral = (
                format(value, "f")
                if isinstance(value, Decimal)
                else str(value)
            )
            return (Word("", "number", (numeral,), "", START),)


def enclosed(value: Listed | Quotation | Dict) -> Enclosure:
    """
    A value written between its delimiters.
    """
    match value:
        case Listed():
            return Enclosure("group", (framed(spread(value.items)),), START)
        case Quotation():
            return Enclosure("quotation", sugared(value.code), START)
        case Dict():
            pairs = tuple(
                item
                for key, v in value.entries
                for item in (named(key), *shown(v))
            )
            return Enclosure("dict", (framed(pairs),), START)
        case _:
            assert_never(value)


def spread(values: tuple[Value, ...]) -> tuple[Item, ...]:
    """
    Values side by side.
    """
    return tuple(item for value in values for item in shown(value))


_desugar_fn = desugar
desugar = Pass("desugar", _desugar_fn)
```

01.02.06 Elaborate

01.02.06 Elaborate

01.02.05-desugar feeds this. 01.02.07-lower is what reifies the core, and the walker has not split that out yet.

```
"""
Elaboration over the core AST: each word's effect inferred from its body
and checked against its effect line, before any line runs.

A sort is what elaboration knows of a value: a number, a text, a symbol,
or a value of no sort it tracks (a list, a quotation, a dict, what a
control word leaves: trusted, hole typed-fragment). While a body is
inferred, the values it takes are inputs whose sorts are not known yet;
arithmetic on one makes it a number. An arrow is a word's inferred
effect: the sorts it takes and leaves. A ? is a goal: reported with the
arrow that would fill it, elaboration going on past it; a _ is inferred
as nothing, or refused (holes goal-placeholder, infer-hole).
"""

from collections.abc import Mapping
from contextlib import suppress
from dataclasses import dataclass, replace
from decimal import Decimal
from enum import Enum, IntEnum
from typing import assert_never

from fpl.arrow import Pass
from fpl.ast_core import (
    EFFECTS,
    Bind,
    Call,
    Define,
    Equal,
    Guarded,
    Inverse,
    Keyed,
    Match,
```

```
    Node,
    Pattern,
    Push,
    Refuse,
    Row,
    Run,
    Statement,
    Strand,
    Symbol,
    Value,
    Var,
    Wild,
)
from fpl.errors import FplError, Span
from fpl.eval import CONTROLS


@dataclass(frozen=True)
class Input:
    """
    The index-th value a word takes, its sort not known yet.
    """

    index: int


class Verdict(IntEnum):
    """
    Whether values of known sorts fit a pattern: surely not, perhaps,
    surely; a conjunction is the least of its parts.
    """

    NO = 0
    MAYBE = 1
    YES = 2


class Kind(Enum):
    """
    A sort of value elaboration tracks; VALUE is one of no sort it
    tracks.
    """
```

```
    NUMBER = "number"
    TEXT = "text"
    SYMBOL = "symbol"
    VALUE = "value"


type Sort = Kind | Input


@dataclass(frozen=True)
class Arrow:
    """
    A word's effect: the sorts it takes, then those it leaves; an Input
    in `outs` is the value taken at that index.
    """

    ins: tuple[Sort, ...]
    outs: tuple[Sort, ...]


@dataclass(frozen=True)
class Dispatch(Arrow):
    """
    A dispatcher's arrow, and its rows in the order it tries them
    (design 09 §3.6).
    """

    rows: tuple[Row, ...]


@dataclass(frozen=True)
class Goal:
    """
    A ? met: where it is and the arrow that would fill it.
    """

    span: Span
    arrow: Arrow


@dataclass(frozen=True)
class Typing:
    """
```

```
    What elaboration knows at a point in the code: the sorts on the
    stack, top last; the sort each bound name stands for; the sorts the
    body's inputs have been found to be; the goals met so far; whether
    the code gets here, which a refusal never passes.
    """

    stack: tuple[Sort, ...]
    env: Mapping[str, Sort]
    known: Mapping[int, Sort]
    goals: tuple[Goal, ...] = ()
    reached: bool = True


class UntypedError(Exception):
    """
    Code outside what elaboration infers (a control word, whose effect
    is the code it runs; a match whose rows disagree): its word's effect
    line is trusted.
    """


class UnderflowError(FplError):
    """
    Fewer values than a word, a binder or a match takes.
    """

    def __init__(self, span: Span) -> None:
        """
        An underflow at the word, binder or match the span points to;
        the message is always the same.
        """
        super().__init__(span, "stack underflow")


HOLES = ("?", "_")
NUMBERS = Arrow((Kind.NUMBER, Kind.NUMBER), (Kind.NUMBER,))
ARROWS: dict[str, Arrow] = {
    **{
        name: Arrow(
            tuple(map(Input, range(len(e.ins)))), (Kind.VALUE,) * len(e.outs)
        )
        for name, e in EFFECTS.items()
        if name not in CONTROLS and name not in HOLES
```

```
    },
    "+": NUMBERS,
    "-": NUMBERS,
    "times": NUMBERS,
    "swap": Arrow((Input(0), Input(1)), (Input(1), Input(0))),
    "dup": Arrow((Input(0),), (Input(0), Input(0))),
    "drop": Arrow((Input(0),), ()),
}
QUERIES = ("history", "doc", "effect")
TESTED = {
    "Int": Kind.NUMBER,
    "Decimal": Kind.NUMBER,
    "Text": Kind.TEXT,
    "Symbol": Kind.SYMBOL,
}


def elaborate(
    statements: tuple[Statement, ...],
) -> tuple[dict[str, Arrow], tuple[Goal, ...]]:
    """
    Every word's arrow, builtins and queries too, and the goals met, in
    the order written. A body that leaves another count than its effect
    line promises is refused at its definition, and a sort a word cannot
    take, in a body or a line, at the word. A word called before its
    last definition is inferred meets that definition's effect line. A
    goal in code that is not typed is not met.
    """
    arrows = dict(ARROWS)
    defines = [s for s in statements if isinstance(s, Define)]
    for d in defines:
        arrows[d.word] = dispatching(d, declared(d))
        arrows.update(
            {
                f"{d.word}/{query}": Arrow((), (Kind.VALUE,))
                for query in QUERIES
            }
        )
    last = {d.word: d for d in defines}
    goals: list[Goal] = []
    for s in statements:
        with suppress(UntypedError):
            goals.extend(checked(s, last, arrows))
```

```
    return arrows, tuple(goals)


def checked(
    statement: Statement, last: Mapping[str, Define], arrows: dict[str, Arrow]
) -> tuple[Goal, ...]:
    """
    The goals a statement meets; a word's last definition sets its
    arrow.
    """
    if isinstance(statement, Run):
        return after(Typing((), {}, {}), statement.code, arrows).goals
    arrow, goals = inferred(statement, arrows)
    if last[statement.word] is statement:
        arrows[statement.word] = dispatching(statement, arrow)
    return goals


def dispatching(define: Define, arrow: Arrow) -> Arrow:
    """
    A dispatcher's arrow with its rows; any other word's as it is.
    """
    match define.code:
        case (Match(rows=rows),) if len(define.clause) == 1:
            return Dispatch(arrow.ins, arrow.outs, rows)
        case _:
            return arrow


def declared(define: Define) -> Arrow:
    """
    The effect line as an arrow: any values in, values of no known sort
    out.
    """
    ins = tuple(map(Input, range(len(define.effect.ins))))
    return Arrow(ins, (Kind.VALUE,) * len(define.effect.outs))


def inferred(
    define: Define, arrows: Mapping[str, Arrow]
) -> tuple[Arrow, tuple[Goal, ...]]:
    """
    The arrow of the body, its inputs as general as the body lets them
```

```
    be, and the goals it meets; the effect line's, and none, if the body
    is untyped.
    """
    start = Typing(tuple(map(Input, range(len(define.effect.ins)))), {}, {})
    promised = len(define.effect.outs)
    try:
        end = after(start, define.code, arrows, promised)
    except UntypedError:
        return declared(define), ()
    if len(end.stack) != promised:
        message = f"leaves {len(end.stack)} values, its effect line {promised}"
        raise FplError(define.span, f"{define.word} {message}")
    return Arrow(
        resolved(start.stack, end), resolved(end.stack, end)
    ), end.goals


def resolved(sorts: tuple[Sort, ...], typing: Typing) -> tuple[Sort, ...]:
    """
    The sorts with each input found to be of a sort replaced by it.
    """
    return tuple(known(s, typing) for s in sorts)


def known(s: Sort, typing: Typing) -> Sort:
    """
    An input's sort once found, else the sort itself.
    """
    return typing.known.get(s.index, s) if isinstance(s, Input) else s


def after(
    typing: Typing,
    code: tuple[Node, ...],
    arrows: Mapping[str, Arrow],
    need: int | None = None,
) -> Typing:
    """
    What is known once the code has run, `need` the count it must end at
    when known; a node that cannot run is refused at its span.
    """
    for index, node in enumerate(code):
        rest = code[index + 1 :]
```

```
        if (
            isinstance(node, Call)
            and node.name in HOLES
            and node.name not in typing.env
        ):
            typing = filled(typing, node, rest, arrows, need)
        else:
            typing = through(typing, node, arrows, None if rest else need)
    return typing


def filled(
    typing: Typing,
    node: Call,
    rest: tuple[Node, ...],
    arrows: Mapping[str, Arrow],
    need: int | None,
) -> Typing:
    """
    A ? takes the stack under it and leaves values of no known sort, as
    many as the rest takes, and a goal with those values' sorts; a _ is
    nothing, when the rest takes the stack as it is, else refused at it.
    """
    arrow = Arrow(
        resolved(typing.stack, typing),
        wanted(typing, node, rest, arrows, need),
    )
    if node.name == "?":
        stack = (Kind.VALUE,) * len(arrow.outs)
        return replace(
            typing, stack=stack, goals=(*typing.goals, Goal(node.span, arrow))
        )
    if len(arrow.outs) != len(arrow.ins):
        raise FplError(node.span, f"cannot infer _ : {shown(arrow)}")
    return typing


def wanted(
    typing: Typing,
    hole: Call,
    rest: tuple[Node, ...],
    arrows: Mapping[str, Arrow],
    need: int | None,
```

```
) -> tuple[Sort, ...]:
    """
    The sorts of the fewest values the rest runs on, ending at `need`
    values when known; a value the rest puts no sort on is of none;
    refused at the hole when no count does.

    Past `reach(rest)` values nothing underflows, and each value more is
    either left on top or cleared by a hole in the rest: `reach(rest) +
    need` values end at `need` when any count does.
    """
    for count in range(reach(rest, arrows) + (need or 0) + 1):
        start = Typing(tuple(map(Input, range(count))), typing.env, {})
        with suppress(UnderflowError):
            end = after(start, rest, arrows, need)
            if need is None or len(end.stack) >= need:
                found = resolved(start.stack, end)
                return tuple(
                    Kind.VALUE if isinstance(s, Input) else s for s in found
                )
    ending = "runs" if need is None else f"ends at {need}"
    raise FplError(
        hole.span, f"cannot infer {hole.name} : no count it leaves {ending}"
    )


def reach(code: tuple[Node, ...], arrows: Mapping[str, Arrow]) -> int:
    """
    The most values code can take from under it: what each node takes,
    summed, a match's most taking row counted; a hole takes none, since
    it takes only what is there.
    """
    total = 0
    for node in code:
        match node:
            case Call():
                total += (
                    len(arrows[node.name].ins) if node.name in arrows else 0
                )
            case Bind():
                total += 1 + reach(node.body, arrows)
            case Match():
                total += len(node.rows[0].patterns)
                total += max(reach(row.body, arrows) for row in node.rows)
```

```
            case Push() | Keyed() | Refuse():
                pass
            case _:
                assert_never(node)
    return total


def shown(arrow: Arrow) -> str:
    """
    An arrow as an effect line: an input as t and its index.
    """
    names = [
        f"t{s.index}" if isinstance(s, Input) else s.value
        for s in (*arrow.ins, *arrow.outs)
    ]
    return " ".join([*names[: len(arrow.ins)], "--", *names[len(arrow.ins) :]])


def reported(goal: Goal) -> str:
    """
    The line a goal is reported as.
    """
    return f"GOAL {goal.span.line}:{goal.span.col} ? : {shown(goal.arrow)}"


def through(
    typing: Typing,
    node: Node,
    arrows: Mapping[str, Arrow],
    need: int | None = None,
) -> Typing:
    """
    What is known once one node has run.
    """
    match node:
        case Push():
            return replace(typing, stack=(*typing.stack, sort(node.value)))
        case Keyed():
            return replace(typing, stack=(*typing.stack, Kind.VALUE))
        case Bind():
            return bound(typing, node, arrows, need)
        case Call():
            return called(typing, node, arrows)
```

```
        case Match():
            return matched(typing, node, arrows, need)
        case Refuse():
            return replace(typing, reached=False)
        case _:
            assert_never(node)


def bound(
    typing: Typing, node: Bind, arrows: Mapping[str, Arrow], need: int | None
) -> Typing:
    """
    The top taken and named for the binder's scope, which runs; the name
    dies with it.
    """
    if not typing.stack:
        raise UnderflowError(node.span)
    env = {**typing.env, node.name: typing.stack[-1]}
    end = after(
        replace(typing, stack=typing.stack[:-1], env=env),
        node.body,
        arrows,
        need,
    )
    return replace(end, env=typing.env)


def matched(
    typing: Typing, node: Match, arrows: Mapping[str, Arrow], need: int | None
) -> Typing:
    """
    Each row run on the values under those the match takes, its pattern
    names standing for values of no known sort; the sorts the rows agree
    on, where they leave as many values, else untyped. Every row counts,
    one after a sure fit included: a sort a row finds for an input holds
    for the rows after it and the word, whichever row runs, and what a
    row that never runs leaves joins the agreement, so no pruning of
    rows can let the word leave a sort it does not promise (hole
    typed-fragment); pruning serves only `chosen`.
    """
    cut = len(typing.stack) - len(node.rows[0].patterns)
    if cut < 0:
        raise UnderflowError(node.span)
```

```
    ends: list[Typing] = []
    for row in node.rows:
        names = dict.fromkeys(bindings(row.patterns), Kind.VALUE)
        begun = replace(
            typing, stack=typing.stack[:cut], env={**typing.env, **names}
        )
        ends.append(after(begun, row.body, arrows, need))
        typing = replace(typing, known=ends[-1].known, goals=ends[-1].goals)
    return replace(typing, stack=agreed(ends))


def agreed(ends: list[Typing]) -> tuple[Sort, ...]:
    """
    The sorts rows leave, inputs resolved by what every row found: each
    where they agree, else of no known sort; untyped where they leave
    different counts. A row that refuses leaves nothing.
    """
    stacks = {resolved(end.stack, ends[-1]) for end in ends if end.reached}
    if len({len(stack) for stack in stacks}) != 1:
        raise UntypedError
    columns = map(set, zip(*stacks, strict=True))
    return tuple(
        column.pop() if len(column) == 1 else Kind.VALUE for column in columns
    )


def bindings(patterns: tuple[Pattern, ...]) -> tuple[str, ...]:
    """
    The names patterns bind, in the order written.
    """
    return tuple(name for pattern in patterns for name in bound_by(pattern))


def bound_by(pattern: Pattern) -> tuple[str, ...]:
    """
    The names one pattern binds.
    """
    match pattern:
        case Var():
            return (pattern.name,)
        case Guarded():
            return bound_by(pattern.pattern)
        case Inverse():
```

```
            return bindings(pattern.args)
        case Wild() | Equal():
            return ()
        case _:
            assert_never(pattern)


def called(typing: Typing, node: Call, arrows: Mapping[str, Arrow]) -> Typing:
    """
    A bound name pushes the sort it stands for; a word takes and leaves
    its arrow's sorts.
    """
    if node.name in typing.env:
        return replace(typing, stack=(*typing.stack, typing.env[node.name]))
    if node.name not in arrows:
        raise UntypedError
    return applied(
        typing, chosen(typing, arrows[node.name], arrows), node.span
    )


def chosen(typing: Typing, arrow: Arrow, arrows: Mapping[str, Arrow]) -> Arrow:
    """
    The arrow a call applies: a dispatcher's resolved to its clause's
    when exactly one row survives, it is a clause's, no value is an
    input and, where the row only may fit, the clause takes the values'
    sorts, since the run would miss where it refuses; else the arrow as
    it is.
    """
    cut = len(typing.stack) - len(arrow.ins)
    if not isinstance(arrow, Dispatch) or cut < 0:
        return arrow
    sorts = resolved(typing.stack[cut:], typing)
    if any(isinstance(s, Input) for s in sorts):
        return arrow
    match surviving(arrow.rows, sorts, arrows):
        case (Row(body=(*_, Call(name=clause))) as row,):
            found = arrows[clause]
            if verdict(row, sorts, arrows) is Verdict.YES or taken(
                found, sorts
            ):
                return found
            return arrow
```

```
        case _:
            return arrow


def surviving(
    rows: tuple[Row, ...], sorts: tuple[Sort, ...], arrows: Mapping[str, Arrow]
) -> tuple[Row, ...]:
    """
    The rows values of the sorts may take, in order: those that surely
    miss dropped, none after the first that surely fits.
    """
    kept: list[Row] = []
    for row in rows:
        fit = verdict(row, sorts, arrows)
        if fit is not Verdict.NO:
            kept.append(row)
        if fit is Verdict.YES:
            break
    return tuple(kept)


def verdict(
    row: Row, sorts: tuple[Sort, ...], arrows: Mapping[str, Arrow]
) -> Verdict:
    """
    How surely values of the sorts fit the row's patterns.
    """
    fits = (
        fitted(have, p, arrows)
        for have, p in zip(sorts, row.patterns, strict=True)
    )
    return min(fits, default=Verdict.YES)


def fitted(
    have: Sort, pattern: Pattern, arrows: Mapping[str, Arrow]
) -> Verdict:
    """
    How surely a value of the sort fits the pattern: a name or _ surely
    does; a guarded pattern, an ascription `x: w` included, at most may,
    and surely not where its pattern or its test surely misses; a
    literal or a constructor may.
    """
```

```
    match pattern:
        case Var() | Wild():
            return Verdict.YES
        case Guarded():
            return min(
                fitted(have, pattern.pattern, arrows),
                tested(have, pattern.test, arrows),
            )
        case Equal() | Inverse():
            return Verdict.MAYBE
        case _:
            assert_never(pattern)


def tested(have: Sort, test: str, arrows: Mapping[str, Arrow]) -> Verdict:
    """
    How surely a test leaves 1 on a value of the sort, never surely: the
    word runs on the value only at run time, so a guarded row is never a
    sure fit and pruning never stops at it. A builtin type word surely
    does not on a known sort other than its own; on its own sort, an
    input or a value of no known sort it may, as any other test may, a
    type word the program redefines included.
    """
    want = TESTED.get(test)
    if want is None or arrows.get(test) is not ARROWS[test]:
        return Verdict.MAYBE
    if isinstance(have, Input) or have in (Kind.VALUE, want):
        return Verdict.MAYBE
    return Verdict.NO


def taken(arrow: Arrow, sorts: tuple[Sort, ...]) -> bool:
    """
    Whether the arrow takes values of the sorts, as `met` finds.
    """
    pairs = zip(arrow.ins, sorts, strict=True)
    return all(
        isinstance(need, Input) or have in (need, Kind.VALUE)
        for need, have in pairs
    )


def applied(typing: Typing, arrow: Arrow, span: Span) -> Typing:
```

```
    """
    The arrow's inputs met by the values under the top, its outputs in
    their place.
    """
    cut = len(typing.stack) - len(arrow.ins)
    if cut < 0:
        raise UnderflowError(span)
    local: dict[int, Sort] = {}
    found = dict(typing.known)
    for need, have in zip(arrow.ins, typing.stack[cut:], strict=True):
        met(need, known(have, typing), local, found, span)
    outs = tuple(
        local[s.index] if isinstance(s, Input) else s for s in arrow.outs
    )
    return replace(typing, stack=(*typing.stack[:cut], *outs), known=found)


def met(
    need: Sort,
    have: Sort,
    local: dict[int, Sort],
    found: dict[int, Sort],
    span: Span,
) -> None:
    """
    One input met: a general one takes the value's sort, a number wants
    one; an input of the body met by a number is found to be one.
    """
    match need:
        case Input():
            local[need.index] = have
        case _ if isinstance(have, Input):
            found[have.index] = need
        case _ if have not in (need, Kind.VALUE):
            raise FplError(span, "arithmetic on a non-number")
        case _:
            pass


def sort(value: Value) -> Sort:
    """
    The sort of a value: a strand has its items' sort when they share
    one.
```

```
    """
    match value:
        case int() | Decimal():
            return Kind.NUMBER
        case str():
            return Kind.TEXT
        case Symbol():
            return Kind.SYMBOL
        case Strand():
            sorts = set(map(sort, value.items))
            return sorts.pop() if len(sorts) == 1 else Kind.VALUE
        case _:
            return Kind.VALUE


def _checked(statements):
    _elaborate_fn(statements)
    return statements


_elaborate_fn = elaborate
elaborate = Pass("elaborate", _checked)
```

01.02.07 Lower

01.02.07 Lower

`Elaborated → Core`

01.02.06-elaborate is the previous arrow. 01.02.08-normalize is the next.

This page is the signature. The walker does not have this pass on its own yet.

01.02.08 Normalize

01.02.08 Normalize

`Core → Core`

01.02.07-lower is the previous arrow. 01.02.09-optimize is the next.

This page is the signature. The walker does not have this pass on its own yet.

01.02.09 Optimize

01.02.09 Optimize

`Core → Core`

01.02.08-normalize is the previous arrow. 01.02.11-codegen is the next.

This page is the signature. The walker does not have this pass on its own yet.

01.02.10 Evaluate

01.02.10 Evaluate

The environment rides inside the call. This wrapper closes over the walker's fuel.

```
"""
The core as a step machine: a state is the stack and the code still to
run, and `step` runs one node of it. Over the core AST only.
"""

from collections import deque
from collections.abc import Callable, Iterator, Mapping
from contextvars import ContextVar
from dataclasses import dataclass
from decimal import Decimal
from functools import partial
from typing import assert_never

import icontract

from fpl.arrow import Pass
from fpl.ast_core import (
    DIGITS,
    EFFECTS,
    Bind,
    Call,
    Define,
    Dict,
    Equal,
    Guarded,
    Inverse,
    Keyed,
    Listed,
    Match,
    Node,
    Number,
    Pattern,
    Push,
```

```
    Quotation,
    Refuse,
    Row,
    Run,
    Statement,
    Strand,
    Symbol,
    Value,
    Var,
    Wild,
)
from fpl.errors import FailError, FplError, Span

type Builtin = Callable[..., tuple[Value, ...]]
type Words = Mapping[str, tuple[Node, ...]]
type Control = Callable[..., tuple[Node, ...]]
type Bound = dict[str, Value]
type Stack = tuple[Value, ...]

LIMIT = 10**DIGITS
"""
The least magnitude a number cannot have.
"""


@dataclass(frozen=True)
class State:
    """
    The stack, top last; the code still to run; the definitions by name.
    """

    stack: tuple[Value, ...]
    code: tuple[Node, ...]
    words: Words


def evaluate(
    statements: tuple[Statement, ...], fuel: int | None = None
) -> tuple[tuple[Value, ...], ...]:
    """
    The stack each run line leaves, every line on a fresh stack and
    within fuel steps, or with no bound when fuel is None; a fuel below
    zero would bound nothing, so it is refused before the first step. A
```

```
    definition is called at its word; a later definition of a word
    shadows an earlier one, for every line; word/history pushes the ones
    it shadows, oldest first, each as a quotation, word/doc the
    docstring of the one in force and word/effect its effect line as a
    list of strings, +fail last when it may fail.
    """
    budget(fuel)
    logged: dict[str, list[Define]] = {}
    for s in statements:
        if isinstance(s, Define):
            logged.setdefault(s.word, []).append(s)
    words = {word: log[-1].code for word, log in logged.items()}
    for word, log in logged.items():
        words[f"{word}/history"] = (
            Push(Listed(tuple(Quotation(d.code) for d in log[:-1]))),
        )
        words[f"{word}/doc"] = (Push(log[-1].doc),)
        words[f"{word}/effect"] = (Push(effect_line(log[-1])),)
    return tuple(
        metered(s, words, fuel) for s in statements if isinstance(s, Run)
    )


def budget(fuel: int | None) -> None:
    """
    Refuses a fuel below zero, which Meter would count down past zero
    and never stop on.
    """
    if fuel is not None and fuel < 0:
        raise ValueError(f"fuel below zero: {fuel}")


@dataclass
class Meter:
    """
    The steps a run line may still take, None for no bound, and the line
    charged for them.
    """

    left: int | None
    span: Span

    def spend(self) -> None:
```

```
        """
        One step taken; refused, at the line, once none is left.
        """
        if self.left == 0:
            raise FplError(self.span, "out of fuel")
        if self.left is not None:
            self.left -= 1


FUEL: ContextVar[Meter] = ContextVar("fuel")
UNMETERED = Meter(None, Span(1, 1))


def metered(run: Run, words: Words, fuel: int | None) -> tuple[Value, ...]:
    """
    The stack the run line leaves, its steps and those of every run
    nested in it charged to one meter. A recursion deeper than the
    walker's own stack is an error at the line.
    """
    token = FUEL.set(Meter(fuel, run.span))
    try:
        return final(State((), run.code, words))
    except RecursionError:
        raise FplError(run.span, "recursion too deep") from None
    finally:
        FUEL.reset(token)


def effect_line(define: Define) -> Listed:
    """
    ins -- outs, a typed input as its two strings `x:` and its type,
    then +fail if the word may fail (hole effect-query).
    """
    effect = define.effect
    fails = ("+fail",) if effect.fails else ()
    typed = zip(effect.ins, effect.types, strict=True)
    ins = (text for name, part in typed for text in slotted(name, part))
    return Listed((*ins, "--", *effect.outs, *fails))


def slotted(name: str, part: str | None) -> tuple[str, ...]:
    """
    An input as its effect line writes it: the name, or `name:` and its
```

```
    type.
    """
    return (name,) if part is None else (name + ":", part)


def final(state: State) -> tuple[Value, ...]:
    """
    The stack once no code is left, each step spent from the line's
    meter, if any.

    A step reads only the first node and leaves the rest of the code
    behind what it puts in place, so each step is taken on its node
    alone and the code still waiting is held here, next node last: a
    step costs the code it puts in place, not the code waiting. Stepping
    the whole state would copy that code each step, quadratic in the
    steps of a recursion that leaves a call waiting each time (`f f`),
    which no budget of a million steps bounds in time (#144). The stack
    and the values a step builds are still copied whole, so a loop that
    grows either each round stays quadratic in its steps.
    """
    meter = FUEL.get(UNMETERED)
    stack, waiting = state.stack, list(reversed(state.code))
    while waiting:
        meter.spend()
        after = step(State(stack, (waiting.pop(),), state.words))
        stack = after.stack
        waiting.extend(reversed(after.code))
    return stack


def running(state: State) -> bool:
    """
    Some code is left to run.
    """
    return bool(state.code)


@icontract.require(running)
def step(state: State) -> State:
    """
    Run the first node: push its value, put a defined word's code in its
    place, apply a builtin to the values it takes, put a binder's scope
    in its place with the top for its name, or push the dict its values
```

```
    build. A control word puts the code it runs in its place, and a
    match the body of the row it chose. A refusal raises its error.
    """
    node, rest = state.code[0], state.code[1:]
    match node:
        case Push():
            return State((*state.stack, node.value), rest, state.words)
        case Bind():
            return binding(node, state, rest)
        case Keyed():
            return State(
                (*state.stack, gathered(node, state.words)), rest, state.words
            )
        case Call() if node.name in state.words:
            return State(
                state.stack, state.words[node.name] + rest, state.words
            )
        case Call():
            return builtin(node, state, rest)
        case Match():
            return matching(node, state, rest)
        case Refuse():
            raise FplError(node.span, node.message)
        case _:
            assert_never(node)


def binding(node: Bind, state: State, rest: tuple[Node, ...]) -> State:
    """
    The binder's scope in its place, the top standing for its name.
    """
    if not state.stack:
        raise FplError(node.span, "stack underflow")
    scope = substitute(node.body, node.name, state.stack[-1])
    return State(state.stack[:-1], scope + rest, state.words)


def substitute(
    code: tuple[Node, ...], name: str, value: Value
) -> tuple[Node, ...]:
    """
    The code with every call of name, in quotations too, a push of
    value; a binder of the same name shadows it for its scope.
```

```
    """
    return tuple(replaced(node, name, value) for node in code)


def replaced(node: Node, name: str, value: Value) -> Node:
    """
    One node with name standing for value.
    """
    match node:
        case Call():
            return Push(value) if node.name == name else node
        case Push():
            return Push(held(node.value, name, value))
        case Bind():
            return rebinder(node, name, value)
        case Keyed():
            return rekeyed(node, name, value)
        case Match():
            return rematched(node, name, value)
        case Refuse():  # pragma: no cover -- a refusing row binds nothing
            return node
        case _:
            assert_never(node)


def rebinder(node: Bind, name: str, value: Value) -> Bind:
    """
    A binder's body with name standing for value, unless it binds name
    anew.
    """
    body = (
        node.body if node.name == name else substitute(node.body, name, value)
    )
    return Bind(node.name, body, node.span)


def rekeyed(node: Keyed, name: str, value: Value) -> Keyed:
    """
    A dict's values with name standing for value.
    """
    return Keyed(
        tuple((key, replaced(n, name, value)) for key, n in node.entries),
        node.span,
```

```
    )


def rematched(node: Match, name: str, value: Value) -> Match:
    """
    A match's rows with name standing for value.
    """
    return Match(
        tuple(rebound(row, name, value) for row in node.rows), node.span
    )


def rebound(row: Row, name: str, value: Value) -> Row:
    """
    A row with name standing for value, up to the pattern that binds
    name anew.
    """
    patterns: list[Pattern] = []
    shadowed = False
    for pattern in row.patterns:
        patterns.append(pattern if shadowed else pinned(pattern, name, value))
        shadowed = shadowed or name in names(pattern)
    body = row.body if shadowed else substitute(row.body, name, value)
    return Row(tuple(patterns), body)


def pinned(pattern: Pattern, name: str, value: Value) -> Pattern:
    """
    A pattern with each $name the value it pins.
    """
    match pattern:
        case Equal():
            return Equal(replaced(pattern.node, name, value))
        case Inverse():
            args = tuple(pinned(p, name, value) for p in pattern.args)
            return Inverse(pattern.name, args, pattern.span)
        case Guarded():
            return Guarded(
                pinned(pattern.pattern, name, value),
                pattern.test,
                pattern.span,
            )
        case Wild() | Var():
```

```
            return pattern
        case _:
            assert_never(pattern)


def names(pattern: Pattern) -> frozenset[str]:
    """
    The names a pattern binds.
    """
    match pattern:
        case Var():
            return frozenset((pattern.name,))
        case Inverse():
            return frozenset[str]().union(*map(names, pattern.args))
        case Guarded():
            return names(pattern.pattern)
        case Wild() | Equal():
            return frozenset()
        case _:
            assert_never(pattern)


def matching(node: Match, state: State, rest: tuple[Node, ...]) -> State:
    """
    The values the match takes replaced by the body of the first row
    matching them, each name a pattern bound standing for its value; no
    row matching raises +fail, a FailError, at the match.
    """
    cut = len(state.stack) - len(node.rows[0].patterns)
    if cut < 0:
        raise FplError(node.span, "stack underflow")
    for row in node.rows:
        bound = matches(row.patterns, state.stack[cut:], state.words)
        if bound is not None:
            code = row.body
            for name, value in bound.items():
                code = substitute(code, name, value)
            return State(state.stack[:cut], code + rest, state.words)
    raise FailError(node.span, "no row matches")


def matches(
    patterns: tuple[Pattern, ...], values: Stack, words: Words
```

```
) -> Bound | None:
    """
    Each pattern against its value, left to right, a name bound by one
    pinned in those after it; None if one fails.
    """
    bound: Bound = {}
    for pattern, value in zip(patterns, values, strict=True):
        for name, earlier in bound.items():
            pattern = pinned(pattern, name, earlier)  # noqa: PLW2901 -- pinned in turn
        got = matched(pattern, value, words)
        if got is None:
            return None
        bound |= got
    return bound


def matched(pattern: Pattern, value: Value, words: Words) -> Bound | None:
    """
    What one pattern binds on a value, or None: a guard is its pattern
    matching and then its word leaving 1 on the value (hole guard-test).
    """
    match pattern:
        case Wild():
            return {}
        case Var():
            return {pattern.name: value}
        case Equal():
            return {} if pattern.node == Push(value) else None
        case Guarded():
            return guarded(pattern, value, words)
        case Inverse():
            return unbuilt(pattern, value, words)
        case _:
            assert_never(pattern)


def guarded(guard: Guarded, value: Value, words: Words) -> Bound | None:
    """
    What the inner pattern binds on the value, if the guard word then
    leaves 1 on it; the word runs only on a value the inner pattern
    matched, so a row whose pattern refuses the value falls through even
    where the word would raise on it.
    """
```

```
    bound = matched(guard.pattern, value, words)
    return bound if bound is not None and tested(guard, value, words) else None


def tested(guard: Guarded, value: Value, words: Words) -> bool:
    """
    The guard's word leaves 1 on the value; its miss is not fitting, and
    any other error in it propagates as that error (design 09 §3.9).
    """
    try:
        left = final(State((value,), (Call(guard.test, guard.span),), words))
    except FailError:
        return False
    return left == (1,)


def unbuilt(pattern: Inverse, value: Value, words: Words) -> Bound | None:
    """
    What the arguments bind on the values the constructor, run
    backwards, gives.
    """
    given = undone(pattern.name, (value,), words, pattern.span)
    if given is None or len(given) != len(pattern.args):
        return None
    return matches(pattern.args, given, words)


def undone(
    name: str,
    stack: Stack,
    words: Words,
    span: Span,
    trail: frozenset[str] = frozenset(),
) -> Stack | None:
    """
    The stack with the word run backwards on its top, or None where the
    top is not a value it builds. A defined word runs its body
    backwards, a primitive its inverse; any other word, or a word inside
    its own body, is refused where the pattern names it.
    """
    if name in words and name not in trail:
        state: Stack | None = stack
        for node in reversed(words[name]):
```

```
            if state is None:
                return None
            state = unrun(node, state, words, span, trail | {name})
        return state
    if name in words or name not in INVERSES:
        raise FplError(span, f"{name} is not invertible")
    return INVERSES[name](stack)


def unrun(
    node: Node, stack: Stack, words: Words, span: Span, trail: frozenset[str]
) -> Stack | None:
    """
    One node of a constructor's body run backwards: a push takes its
    value back off.
    """
    match node:
        case Push():
            return unpush(node.value, stack)
        case Call():
            return undone(node.name, stack, words, span, trail)
        case _:
            raise FplError(
                span, "a constructor is words and literals: not invertible"
            )


def unpush(value: Value, stack: Stack) -> Stack | None:
    """
    The stack below its top, when the top is the value.
    """
    match stack:
        case (*below, top) if top == value:
            return tuple(below)
        case _:
            return None


def unswap(stack: Stack) -> Stack | None:
    """
    swap is its own inverse.
    """
    match stack:
```

```
        case (*below, x, y):
            return (*below, y, x)
        case _:
            return None


def unpair(stack: Stack) -> Stack | None:
    """
    A pair's two items.
    """
    match stack:
        case (*below, Listed(items=(a, b))):
            return (*below, a, b)
        case _:
            return None


def uncons(stack: Stack) -> Stack | None:
    """
    A non-empty list's first item and the rest.
    """
    match stack:
        case (*below, Listed(items=(x, *xs))):
            return (*below, x, Listed(tuple(xs)))
        case _:
            return None


INVERSES: dict[str, Callable[[Stack], Stack | None]] = {
    "swap": unswap,
    "pair": unpair,
    "cons": uncons,
}


def held(inner: Value, name: str, value: Value) -> Value:
    """
    A pushed value with name standing for value in the code it holds.
    """
    match inner:
        case Quotation():
            return Quotation(substitute(inner.code, name, value))
        case Listed():
```

```
            return Listed(
                tuple(held(item, name, value) for item in inner.items)
            )
        case _:
            return inner


def gathered(node: Keyed, words: Words) -> Dict:
    """
    Each value run on a fresh stack, where it must leave one value.
    """
    entries: list[tuple[str, Value]] = []
    for key, code in node.entries:
        stack = final(State((), (code,), words))
        if len(stack) != 1:
            raise FplError(node.span, "a dict value is one value")
        entries.append((key, stack[0]))
    return Dict(tuple(entries))


def builtin(call: Call, state: State, rest: tuple[Node, ...]) -> State:
    """
    The state with a builtin's results in place of its arguments, or
    with a control word's code before the rest. Fewer values than it
    takes is refused at the word: an effect line promised more than its
    body left.
    """
    cut = len(state.stack) - len(EFFECTS[call.name].ins)
    if cut < 0:
        raise FplError(call.span, "stack underflow")
    below, taken = state.stack[:cut], state.stack[cut:]
    if call.name in CONTROLS:
        code = CONTROLS[call.name](call.span, state.words, *taken)
        return State(below, code + rest, state.words)
    return State(
        (*below, *BUILTINS[call.name](call.span, *taken)), rest, state.words
    )


def operands(value: Value, span: Span) -> tuple[Number, ...]:
    """
    A number as one operand, a strand of numbers as its items; anything
    else is refused.
```

```
    """
    items = value.items if isinstance(value, Strand) else (value,)
    numbers = tuple(item for item in items if isinstance(item, int | Decimal))
    if len(numbers) < len(items):
        raise FplError(span, "arithmetic on a non-number")
    return numbers


def arithmetic(
    op: Callable[[Number, Number], Number], span: Span, a: Value, b: Value
) -> tuple[Value, ...]:
    """
    A pervasive operator: a number meets a number, or each item of a
    strand; two strands meet item by item and must be as long. A result
    of more than DIGITS digits before its point is refused at the word.
    """
    xs, ys = operands(a, span), operands(b, span)
    if len(xs) != len(ys) and 1 not in (len(xs), len(ys)):
        raise FplError(span, "strands of unequal length")
    zs = tuple(
        op(xs[i % len(xs)], ys[i % len(ys)])
        for i in range(max(len(xs), len(ys)))
    )
    if any(abs(z) >= LIMIT for z in zs):
        raise FplError(span, "number too large")
    return (
        Strand(zs)
        if isinstance(a, Strand) or isinstance(b, Strand)
        else zs[0],
    )


def swap(_span: Span, a: Value, b: Value) -> tuple[Value, ...]:
    """
    x y -- y x
    """
    return b, a


def dup(_span: Span, a: Value) -> tuple[Value, ...]:
    """
    x -- x x
    """
```

```
    return a, a


def drop(_span: Span, _a: Value) -> tuple[Value, ...]:
    """
    x --
    """
    return ()


def enclose(_span: Span, a: Value) -> tuple[Value, ...]:
    """
    x -- [x]
    """
    return (Quotation((Push(a),)),)


def join(span: Span, a: Value, b: Value) -> tuple[Value, ...]:
    """
    Two quotations as one, or two lists as one.
    """
    match a, b:
        case Quotation(), Quotation():
            return (Quotation(a.code + b.code),)
        case Listed(), Listed():
            return (Listed(a.items + b.items),)
        case _:
            raise FplError(span, ", joins two quotations or two lists")


def pair(_span: Span, a: Value, b: Value) -> tuple[Value, ...]:
    """
    a b -- p : a two-item list (hole pair-shape).
    """
    return (Listed((a, b)),)


def cons(span: Span, x: Value, xs: Value) -> tuple[Value, ...]:
    """
    x xs -- ys : x before the items of a list.
    """
    if not isinstance(xs, Listed):
        raise FplError(span, "cons takes a list")
```

```
    return (Listed((x, *xs.items)),)


def typed(kind: type[object], _span: Span, value: Value) -> tuple[Value, ...]:
    """
    x -- b : 1 when x is of the type, or is a strand whose items all
    are; 0 on any other value, an empty strand included.
    """
    items = value.items if isinstance(value, Strand) else (value,)
    return (
        int(bool(items) and all(isinstance(item, kind) for item in items)),
    )


def unfilled(span: Span) -> tuple[Value, ...]:
    """
    -- : a goal left in the code, refused where it runs (hole
    goal-placeholder).
    """
    raise FplError(span, "unfilled goal")


def nothing(_span: Span) -> tuple[Value, ...]:
    """
    -- : a _ the elaborator inferred as nothing (hole infer-hole).
    """
    return ()


BUILTINS: dict[str, Builtin] = {
    "+": partial(arithmetic, lambda x, y: x + y),
    "-": partial(arithmetic, lambda x, y: x - y),
    "times": partial(arithmetic, lambda x, y: x * y),
    "swap": swap,
    "dup": dup,
    "?": unfilled,
    "_": nothing,
    "drop": drop,
    "enclose": enclose,
    ",": join,
    "pair": pair,
    "cons": cons,
    "Int": partial(typed, int),
```

```
    "Decimal": partial(typed, Decimal),
    "Text": partial(typed, str),
    "Symbol": partial(typed, Symbol),
}


def quoted(value: Value, span: Span) -> Quotation:
    """
    A value that must be a quotation.
    """
    if not isinstance(value, Quotation):
        raise FplError(span, "a quotation is expected")
    return value


def force(span: Span, _words: Words, q: Value) -> tuple[Node, ...]:
    """
    q -- : q run in place; q fills a thunk slot, as every control word's
    quotation does.
    """
    return quoted(q, span).code


def choose(
    span: Span, _words: Words, c: Value, t: Value, e: Value
) -> tuple[Node, ...]:
    """
    c t e -- : t run in place on 1, e on 0; any other condition is
    refused (hole truth-values).
    """
    branches = quoted(e, span), quoted(t, span)
    if not isinstance(c, int) or c not in (0, 1):
        raise FplError(span, "if takes 0 or 1")
    return branches[c].code


def commute(
    span: Span, _words: Words, x: Value, y: Value, q: Value
) -> tuple[Node, ...]:
    """
    x y q -- : q run in place on y x.
    """
    return (Push(y), Push(x), *quoted(q, span).code)
```

```
def repeat(span: Span, _words: Words, q: Value, n: Value) -> tuple[Node, ...]:
    """
    q n -- : q run in place n times; n a count. Each round puts back one
    q and the rest of the count, so the code grows by one q, not n.
    """
    code = quoted(q, span).code
    if not isinstance(n, int) or n < 0:
        raise FplError(span, "repeat takes a count")
    if n == 0:
        return ()
    return (*code, Push(q), Push(n - 1), Call("repeat", span))


def items(xs: Value, span: Span) -> tuple[Value, ...]:
    """
    The items of a strand or a list; anything else is refused.
    """
    if not isinstance(xs, Strand | Listed):
        raise FplError(span, "a strand or a list is expected")
    return xs.items


def single(state: State, span: Span) -> Value:
    """
    The one value a step leaves on its fresh stack (hole step-stack).
    """
    stack = final(state)
    if len(stack) != 1:
        raise FplError(span, "each step leaves one value")
    return stack[0]


def rebuilt(xs: Value, values: list[Value]) -> Push:
    """
    The results pushed as a strand when xs was one and every result is a
    number or a string, else as a list (hole sequence-shape).
    """
    atoms = tuple(v for v in values if isinstance(v, int | Decimal | str))
    if isinstance(xs, Strand) and len(atoms) == len(values):
        return Push(Strand(atoms))
    return Push(Listed(tuple(values)))
```

```
def running_results(
    span: Span, words: Words, xs: Value, q: Value
) -> Iterator[Value]:
    """
    The first item, then q run on the result so far and the next item,
    for each item; one at a time, so a fold need not hold the ones it
    has passed.
    """
    code = quoted(q, span).code
    values = items(xs, span)
    if not values:
        return
    result = values[0]
    yield result
    for x in values[1:]:
        result = single(State((result, x), code, words), span)
        yield result


def each(span: Span, words: Words, xs: Value, q: Value) -> tuple[Node, ...]:
    """
    xs q -- ys : q run on each item alone.
    """
    code = quoted(q, span).code
    return (
        rebuilt(
            xs,
            [single(State((x,), code, words), span) for x in items(xs, span)],
        ),
    )


def scan(span: Span, words: Words, xs: Value, q: Value) -> tuple[Node, ...]:
    """
    xs q -- ys : every result a fold passes through.
    """
    return (rebuilt(xs, list(running_results(span, words, xs, q))),)


def fold(span: Span, words: Words, xs: Value, q: Value) -> tuple[Node, ...]:
    """
```

```
    xs q -- x : the last result of the scan; no item to start from is
    refused.
    """
    last = deque(running_results(span, words, xs, q), maxlen=1)
    if not last:
        raise FplError(span, "fold over nothing")
    return (Push(last[0]),)


CONTROLS: dict[str, Control] = {
    "!": force,
    "if": choose,
    "swap-args": commute,
    "repeat": repeat,
    "each": each,
    "scan": scan,
    "fold": fold,
}


def _run(statements):
    return _eval_fn(statements, 1_000_000)


_eval_fn = evaluate
evaluate = Pass("eval", _run)
```

01.02.11 Codegen

01.02.11 Codegen

`Core → Target`

01.02.09-optimize is the previous arrow. 01.02.10-evaluate is the next.

This page is the signature. The walker does not have this pass on its own yet.

01.03.01 Source

01.03.01 Source

The program text, after the ingress and before lex.

01.03.02 Tokens

01.03.02 Tokens

What the lexer Lark yields. The parser Lark reads these and does not lex again.

01.03.03 Syntax

01.03.03 Syntax

The surface ABT.

01.03.04 Resolved

01.03.04 Resolved

Binders are identities.

01.03.05 Expanded

01.03.05 Expanded

Sugar is gone. The term is still surface.

01.03.06 Elaborated

01.03.06 Elaborated

Polarity, effects, and metavariables are known. Still surface-adjacent.

01.03.07 Core

01.03.07 Core

The dependent-CBPV core. It later splits into Core.Value and Core.Comp. This trial keeps one sort.

01.04.01 Pass

01.04.01 Pass

Python has no pipe operator. >> on a Pass is composition. On an int it stays a shift.

```
from __future__ import annotations

from collections.abc import Callable
from dataclasses import dataclass


@dataclass(frozen=True)
class Pass[A, B]:
    """
    One arrow. Calling it runs the function. >> builds the next arrow.
    """

    name: str
    fn: Callable[[A], B]

    def __call__(self, value: A) -> B:
        return self.fn(value)

    def __rshift__[C](self, nxt: Pass[B, C]) -> Pass[A, C]:
        return Pass(f"{self.name}>>{nxt.name}", lambda value: nxt(self(value)))
```

01.04.02 Run

01.04.02 Run

```
run = lex >> parse >> resolve >> expand >> elaborate >> lower >> evaluate
```

The executable chain is the prefix the walker actually runs. `desugar` stands in for resolve, expand, and lower.

```
from fpl.ast_core import Run
from fpl.desugar import desugar, listing
from fpl.elaborate import elaborate
from fpl.errors import FplError
from fpl.evaluate import evaluate
from fpl.lex import lex
from fpl.parse import parse
from fpl.print import render

run_now = lex >> parse >> desugar >> elaborate >> evaluate

INDENT_COLUMNS = 4


def _notebook_indent(source: str) -> str:
    """
    Turn leading spaces into tabs for notebook editors that indent with
    spaces.

    Uses ceiling division by INDENT_COLUMNS so two spaces still become
    one tab.
    """
    lines: list[str] = []
    for line in source.split("\n"):
        n = 0
        while n < len(line) and line[n] == " ":
            n += 1
        if n == 0:
            lines.append(line)
            continue
```

```
        levels = (n + INDENT_COLUMNS - 1) // INDENT_COLUMNS
        lines.append("\t" * levels + line[n:])
    return "\n".join(lines)


def _run(source: str, session: str) -> str:
    """
    Render one program. `session` is accepted and not stored.
    """
    del session
    return render(listing(run_now(_notebook_indent(source))))


class Session:
    """
    A named run. The name is kept and not stored beyond the object.
    """

    def __init__(self, name: str) -> None:
        self.name = name

    def file(self, path: str) -> str:
        """
        Read a file as UTF-8 and render it.
        """
        source = open(path, encoding="utf-8").read()
        return _run(source, self.name)

    def src(self, source: str) -> str:
        """
        Render a source string.
        """
        return _run(source, self.name)


default = Session("default")


def run_file(path: str) -> int:
    """
    Run one file the way the seed walker does. Goals go to stderr.
    """
    try:
```

```
    text = default.file(path)
except FplError as error:
    print(error)
    return 1
print(text, end="")
return 0
```

01.04.03 Inspect

01.04.03 Inspect

```
inspect = lex >> parse >> resolve >> expand >> elaborate
```

Elaboration establishes the semantic information. Lowering reifies it.

01.04.04 Build

01.04.04 Build

```
build = lex >> parse >> resolve >> expand >> elaborate >> lower >> optimize >> codegen
```

A `>>` chain orders passes. It is not the semantics.

01.05.01 Pipeline

01.05.01 Pipeline

One cell is the in-process chain. The other is the same passes across FON.

```
import fpl

run_now = fpl.lex >> fpl.parse >> fpl.desugar >> fpl.elaborate >> fpl.evaluate
run_now
```

01.06.01 Advance

01.06.01 Advance

Nagini checks this. It does not call Lark, and it does not override a dunder.

```
from dataclasses import dataclass

from nagini_contracts.contracts import Ensures, Requires, Result


@dataclass(frozen=True)
class Span:
    """
    A source position. Nagini sees this. The parser does not live here.
    """

    line: int
    col: int


def column(span: Span) -> int:
    """
    The column, unchanged. The contracts are the whole proof.
    """
    Requires(span.line >= 1)
    Requires(span.col >= 1)
    Ensures(Result() == span.col)
    return span.col
```

2 Docs

DESIGN.md — the FPL vision

> **Status: TEMPLATE.** This is drawn out with the maintainer in a dedicated interview (`.claude/commands/design-fpl.md` / `/design-fpl`). Until the sections below hold real content, **agents must not invent language semantics** — build only what a `features/<name>/spec.md` authored by the maintainer asks for.
>
> This file is edit-protected: changing it requires the literal `[spec]` marker in the commit message (`.githooks/commit-msg`, `.claude/hooks/guard-specs.sh`).

1. One sentence

What is FPL, in a sentence the maintainer would say to another programmer?

TBD

2. Why it exists

The itch. What existing languages get wrong or make hard, that FPL is a reaction to.

TBD

3. Feel

A paragraph of prose, then a short program that captures the intended feel — even if nothing runs it yet.

TBD

4. Non-goals

What FPL is deliberately not. The features it will decline. The audiences it won't chase.

TBD

5. Influences

Languages, papers, tools — what to borrow from each, and what to leave.

TBD

6. Core shape (feeds the stable core AST)

Values, evaluation model (eager/lazy), typing discipline, effects, error model, module model. This is the part `docs/CONVENTIONS.md`*'s* `ast_core.py` *must stay faithful to.*

TBD

7. Open questions

Things genuinely undecided, with the options on the table.

Which accelerator, and can they be one mechanism

The stated working principle is that the project should accelerate itself (`02-docs/02.04-notes/02.04.03-mediating-project.md`). Three candidate accelerators are on the table. Each is real, each has a cost, and every one of those costs lands on the same principle.

Liveness. A live image shortens the loop between changing something and seeing it to almost nothing, which is probably the largest single factor early on. Smalltalk and Factor are the precedents. The cost is reproducibility: an image accumulates state that cannot be reconstructed from source, which runs against a record that travels with the repository (`docs/TASKS.md`, `docs/WORKFLOW.md` rule 7).

Brevity. Two different things get called this and they should not be confused. Semantic brevity is implicit iteration and a small set of very general combinators — APL, J and K do not write loops because their primitives work over whole arrays. Typographic brevity is single-glyph names, which saves typing, and typing is not the bottleneck. Golfing languages such as Vyxal sit at the far end and optimise for bytes in a one-shot solution. The cost is that concatenative and tacit code is hard to read back, including by its author, and velocity over months is mostly a re-reading problem.

Correctness by construction. Refinement types and type-driven development slow the first week and are meant to hold the pace as a codebase grows. Combining refinement-strength types with gradual typing is an existing line of work rather than a wish: Lehmann and Tanter, *Gradual Refinement Types*, POPL 2017; Vazou and others, *Gradual Liquid Type Inference*.

The cost is measured and it is not small. Takikawa and others (*Is Sound Gradual Typing Dead?*, POPL 2016) ran twelve programs in every combination of typed and untyped modules and found that typing a single module could cost 88x, with other implementations at 10x (Reticulated Python) and 22x (Safe TypeScript); under a 3x threshold almost no partial configuration was deployable. That is why unsound erasure won in practice. The result is contested and has improved since — the follow-ups are titled "only mostly dead", "nominally alive and well" and "Corpse reviver" — so this is a live front rather than a closed door. The cost sits at the boundary between typed and untyped code, so the boundary discipline is the thing to design, not the type system.

The question is not which one to pick. It is whether they can be a single mechanism, because a system that is total *and* many-mechanismed is the shape that historically dies.

Which extension mechanism

Related and prior to the above. Forth, Lisp and Smalltalk give three different answers to "how does a user extend the language": the compiler as a library (`CREATE … DOES>`, immediate words, a dictionary that is open at runtime), code as data, and objects in a live image. Taking all three means three mechanisms.

Factor is the existence proof that some of this composes: it is concatenative, has an object system of tuple classes as a central part, a Smalltalk-style image, a self-hosted optimising compiler and an interactive environment, and it got there by making concatenation the one mechanism and growing the object system out of the language rather than beside it. Object systems on Forth are likewise small enough to fit in a few pages (Gforth's Mini-OOF; Pountain, *Object-Oriented Forth*, 1987). Nothing here shows that the third accelerator joins them.

Once §§1–6 are real, promote the settled semantics into `02-docs/02.02-spec.md` (grammar, evaluation, type system) and keep this file as the "why".

SPEC.md — the FPL language specification

> **Status: SKELETON.** Sections fill in as semantics settle — promoted here from `02-docs/02.01-design.md` once decided, and from maintainer-authored `features/<name>/spec.md` as features land. This file is the normative reference; `02-docs/02.01-design.md` holds the *why*.
>
> Edit-protected: changing it requires the `[spec]` marker in the commit message.

1. Lexical structure

TBD — defined by `fpl/grammar.lark` terminals once it exists.

2. Grammar

TBD — `fpl/grammar.lark` is the source of truth; this section summarises and links it.

3. Core language

TBD — the small stable core the surface syntax desugars to (see `docs/CONVENTIONS.md`).

4. Evaluation

TBD — evaluation order, values, environment model.

5. Type system

TBD — discipline, inference (if any), error model.

6. Errors

TBD — every user-facing error is `ERROR: <line>:<col> <message>`; format specified here.

02.03 ROADMAP.md — milestones

Direction only. Scope and ordering shift as `02-docs/02.01-design.md` fills in. Each milestone lands as a stack of small commits (`.agents/WORKFLOW.md`) inside a worktree and is "done" only when `make check` is green (`.agents/CONVENTIONS.md`).

Before milestone 0

The table below starts at a worktree skeleton, but three phases come first and none of them is language work. They are listed so the roadmap does not read as though the lexer begins tomorrow.

1. **The version control harness.** `.harness/scripts/`, `.githooks/`, `.agents/WORKFLOW.md` and the task graph of `.agents/TASKS.md`. Largely built; the store exists and holds nothing yet.
2. **The noslop harness.** `.harness/quality/` and the gate of `.agents/QUALITY.md`: every check, and a bad example each one must refuse, built before there is code to judge, so the pressure is there from the first commit. Built with milestone 0's skeleton, which is its first subject.
3. **Prototyping**, which is where milestone 0 begins.

`02-docs/02.04-notes/02.04.02-sober.md` applies to both harness phases: a harness change earns its place by making a reviewed, landed change cheaper, and there are no language changes yet for either of them to make cheaper. That is a thing to watch, not a rule against them.

#	Milestone	Done when
0	**Worktree skeleton** — `01-bootstrap/pyproject.toml`, `01-bootstrap/Makefile` (`include ../.harness/quality/noslop.mk`), `01-bootstrap/fpl/` module stubs, `03-fpl/03.01-features/_template/`, empty conformance runner	`make ready` runs and passes with zero features

#	Milestone	Done when
1	**Lexer + parser** — `grammar.lark`, `parse.py` → surface AST with spans; `test_ambiguity.py` green	`03-fpl/03.01-features/arithmetic/` parses; zero Earley ambiguity
2	**Tree-walking eval** — `ast_core.py`, `desugar.py`, `eval.py` for the arithmetic core; `python -m fpl file.fpl` runs	`03-fpl/03.01-features/arithmetic/` conformance passes end to end
3	**Errors** — `errors.py`: every failure is `ERROR: <line>:<col> <message>` with a caret underline; no traceback escapes	error-case `.expected` files pass
4	**Desugar layer proven** — a second feature (e.g. `let`, `if`) added as grammar + desugar only, core AST unchanged	new feature green without touching `eval.py`
5	**Types** — `types.py`: a real check over core AST; type-error `.expected` cases	typed conformance passes; pyright still strict-clean
6	**Core IR** — lower core AST → small IR; `eval` runs on IR	conformance unchanged through the IR
7	**Backend: IR → C → clang** — `fpl build file.fpl` emits C, `clang` compiles it; same outputs as the interpreter	conformance suite passes against the compiled binary
8	**Editor tooling** — a tree-sitter grammar for highlighting / structural editing (separate from `grammar.lark`, not the semantic frontend)	grammar parses the `examples/` corpus

Explicitly deferred: an LLVM backend (revisit only after milestone 7 is stable and there's a concrete reason), a language server, a package system, self-hosting.

02.04.01 2026-09-19 — issue, task, patch

A proposal, not a decision. None of it is implemented. If it is accepted it moves into `.agents/TASKS.md` and this note goes away; if it is not, the note records why.

Three objects, and the work each one does

"Issue" carries its meaning from the courtroom rather than from the bug tracker. The legal sense is the older one — the point put forward to be determined — and it comes from *exire*, to go out. That is what an issue does: it puts a matter forward. It says nothing about when the matter is dealt with, or in what order relative to other matters.

	carries	does not carry
`xyz.radicle.issue`	the matter, and the discussion about it	order, dependencies
`nl.psjs.dagtaak`	order, state, assignee, and a pointer to an issue	discussion
`xyz.radicle.patch`	the change, its revisions, the verdicts on them	—

A task points at an issue. One or more patches are offered against it. A patch that is accepted and lands satisfies the task; one that is rejected sends it back for another revision. That loop already lives inside the patch object, per revision and per reviewer, so the task does not model review and never grows comments of its own. Everything that is conversation stays in the issue, and the existing tooling — `rad issue`, `rad patch`, the TUI, the desktop app — keeps working on it unchanged.

This is the whole reason the custom type stays small. It adds the one thing Radicle has no place for, which is the edge between two pieces of work.

Why there is no plan object

A plan proposes to add tasks and the edges between them. But a collaborative object *is* an operation log: every operation is already a delta on the graph, signed by whoever made it. So a plan is not a new kind of object — it is a bundle of operations that has not been applied yet. Two things separate such a bundle from a bare operation: the grouping, and the not-yet-applied state. The first is an edge, described below. The second is what a patch already is.

The question the idea was reaching for is a different one and is not needed to build any of this: when are two decompositions of the same work the same plan?

Two kinds of edge

`blockedBy` is order: this cannot start until that is finished. It is the edge `ready` walks, and the edge a loop is a loop in.

`partOf` is containment: this is one of the pieces that together make up that. `ready` never walks it.

Putting both in one relation leaves a graph that still parses but cannot answer either question. Asking what belongs to a feature returns things that merely precede it; asking what can start returns containers. A container has no work of its own, so it is never ready, and it blocks nothing — what blocks is its contents.

A direction — the thing that is not yet a task, "zou het niet cool zijn als" — is then the top of a `partOf` tree rather than a column on a board. It sits in `list` and never in `ready`, for the same reason any container does.

Depth and width, not a calendar

A Gantt chart is three things stacked: dependencies, decomposition, and a time axis from which start dates, float and the critical path follow. The first two are the two edges above. The third does not transfer, because a duration estimate for agent work is not a real number: a task takes four minutes, or it fails, or it costs three rounds of review because the first attempt solved the wrong problem. A chart drawn from invented durations is not a plan, it is a picture that then gets planned against.

What survives without a single date is the useful half. The critical path is the longest chain through the graph, and it can be counted in steps. The width is how many tasks are ready at once, which is the answer to how many agents can usefully work here before they are waiting on each other. Both follow from `blockedBy` alone.

Shape Up's hill chart is the same move made earlier: it places each scope on a slope from unknown to known, with no estimates and no percentages, and reads progress from whether a dot moves between snapshots rather than from a number.

The time axis can come back afterwards. Every operation is timestamped and signed, so the real sequence can be drawn once the work is done. That is a record of what happened, not a forecast, and it is the only honest Gantt here.

Open

- A direction with no contents yet is a leaf, and by the rule above a leaf is ready. Either containers are marked as such, or the `idea` state comes back to do that job.
- Whether a task may point at more than one issue.
- Whether the patches offered against a task are recorded on the task or read from the patches.

02.04.02 2026-09-19 — plumbing, not prophecy

A standing instruction about tone and about the size of the harness, written down on the date the harness got most of its parts.

What exists on this date

Git hooks, a commit linter, a Nix flake, some bash scripts and a task reducer of a few hundred lines of Python. No parser, no evaluator, no language. Years of work lie between this and any larger vision, and well-turned paragraphs do not shorten them by a day.

For agents

- Do not hype. When the long-term shape of the project comes up, say what exists and what does not.
- You read this repository's docs and notes, and a session may carry memory besides. Handing the maintainer's own framework back to him in polished sentences reads as confirmation from outside and is not. Say so when you do it.
- The ideas here stand in a tradition — Kay's Dynabook, Nelson's *Computer Lib*, Illich's conviviality, by which a tool is one its user can inspect and bend rather than one that prescribes what may be done with it — which is a steadier place to work from than a revelation.
- The harness is where the risk is highest. Tooling for agents is quick to generate, feels like progress, and can grow without bound while the language stands still. Ronacher (below) names the signs: changes that take minutes to make and hours to review, docs that are themselves slop, token burn with no way to tell good work from bad, orchestration layers of hundreds of thousands of lines. A harness change earns its place by making a reviewed, landed change cheaper. Otherwise it waits.

Reference

Armin Ronacher, "Agent Psychosis: Are We Going Insane?", 2026-01-18, https://lucumr.pocoo.org/2026/1/18/agent-psychosis/. The argument about agent infrastructure as found: it

names Gas Town and Beads. The same argument is sometimes attributed to DHH; that could not be confirmed. In Lex Fridman Podcast #501 (00:37:37) he uses "psychosis" the other way round, for failing to recognise how much has changed.

02.04.03 2026-09-20 — the project that mediates the language

A note on one choice that has to be made before it can be made well, kept here so it is not made by accident.

The pattern

Languages that work tend to have had a project that mediated them while they were being built. C had Unix. Rust had a browser engine at Mozilla. Go had Google's server infrastructure. Erlang had Ericsson's telephone switches. Smalltalk had the working environment it was written to run: an editor, windows, a live image you could open while it ran.

The instructive case is the one without a single outside project. Haskell's mediator ended up being its own compiler, and it shows: it is superb at compilers and somewhat awkward everywhere else.

What the mediator actually does

Not requirements. Tilt.

The mediating project decides what the language ends up being *good* at, and that never washes out. C is good at what an operating system needs — bytes, addresses, a thin layer over the machine — and its weaknesses are the same fact seen from the other side. Go is good at what a large server fleet needs, and correspondingly blunt where that was not the question. Erlang is good at what a switch needs, isolation and hot upgrade and failing fast, and strange outside it.

So choosing the mediating project is choosing a permanent bias. It is not a scheduling decision and it cannot be revisited later by refactoring, because the bias does not live in a module. It lives in what the language makes cheap.

Why that matters here in particular

This repository's ambition is a language that is also an environment, in the line of Smalltalk, the Lisp machines and Forth. If that is the target, the mediating project has to exercise the environment and not the compiler.

A version control system, which is otherwise a tempting first thing to self-host, exercises almost the wrong half: it is I/O and data structures with very little computation, so it would tilt the language toward content-addressed storage and plumbing and tell us nothing about the parts a live environment needs. An editor, a shell, a window onto the running system — those are what shaped Smalltalk, and they ask the questions this project is eventually about.

And if there is no outside project at all

The maximal position is to build all of it: language, compiler, environment, tools, the version control underneath — a refusal of what is not invented here, applied to itself. That is not a new idea and it is not obviously a mistake. Oberon did it. Forth did it, often down to the metal. Plan 9 did it. Smalltalk and the Lisp machines did it. TempleOS did it alone.

What the survivors have in common is that they bought totality with one unifying mechanism and a great deal of cutting. Wirth's rule for Oberon was that the whole system had to be comprehensible by one person, and everything that did not fit was removed. Plan 9 made everything a file. Smalltalk sent messages and did nothing else. Forth's answer was to make the language almost nothing at all. What died are the projects that wanted totality *and* richness. The hazard is not the scale; it is the number of distinct mechanisms.

The second cost is the one this note began with. Build everything, and there is no mediating project outside the language: the system mediates itself. That is the Haskell situation carried to its limit — a closed loop which can be perfectly consistent with its own demands while saying nothing about whether it is good for anything else. TempleOS is the monument to that, and it is a monument rather than a tool for exactly this reason.

The move the tradition found against it is small: keep one user outside the loop, using the thing for something the system itself does not need. Unix had real users. Smalltalk had children and a research group who wanted things the implementors did not. One is enough.

Method, and one principle with a precedent

The intended method is conceptual plundering in the manner of Deleuze and Guattari: take the tradition apart, recombine it, put the pieces down somewhere they did not come from, and then see what actually works together and what only looked like it would. Hauntology is the

register — the material is a history of computing that did not happen, handled as something to work with rather than to restore.

The working principle is *sustained velocity*: the project has to accelerate itself, by bootstrapping and dogfooding, so that the thing being built keeps making the building cheaper.

That principle has a name and a precedent inside the very tradition being invoked. Engelbart called it bootstrapping and gave it three levels: A is the work itself, B is improving how the work is done, and C is improving how the improving is done. Running all three at once is the method, and it is how his lab built NLS and the mouse — with the tools they were building, while they were building them. The aim here is the same aim: raise what one ordinary person can do.

The tension this leaves

Those two conclusions pull against each other, and both are in the plan.

Sustained velocity says to spend effort on the system making itself better at making itself. Keeping a user outside the loop says that effort spent inside the loop tells you nothing about whether any of it is worth having. Every hour that goes into the bootstrap is an hour not spent on the one outsider who would answer that.

Neither should quietly win by default. Writing both down is how that gets noticed.

What this does not license

Nothing yet. `02-docs/02.01-design.md` has no content and there is no grammar, and a mediating project is meaningless before there is a language to mediate. The order is: settle on a grammar first; the rest of this note is what to weigh once something runs.

It is written down now only because the choice is easy to make inattentively — by picking up whatever is convenient to build next — and it is exactly the kind of decision this project has said it does not want to be haunted by.

3.1 Features

draft1 — spec

Source: handoff/SHAR-syntax.org, “Draft 1 — the language in itself”; design doc, Draft 1

Why: every example in the syntax documents, in the name spelling (S32), tabs, =;= comment levels (S33), =#= symbols (S29), =$= unquote (S30), =?= goals and =_= inference holes (S40). Each parses under the grammar; the harness reads them from =gram/snips/=. Draft 4 is the pattern-matching sketch (S35–S41). Rung 2 (parsed by the same grammar); their meaning is rung 5, prose, until the elaborator exists.

Claims: D1.1, D1.2, D1.3, D1.4, D1.5, D1.6, D1.7, D1.8, D1.9, D1.10

draft2 — spec

Source: handoff/SHAR-syntax.org, “Draft 2 — examples”; design doc, Draft 2

Why: every example in the syntax documents, in the name spelling (S32), tabs, =;= comment levels (S33), =#= symbols (S29), =$= unquote (S30), =?= goals and =_= inference holes (S40). Each parses under the grammar; the harness reads them from =gram/snips/=. Draft 4 is the pattern-matching sketch (S35–S41). Rung 2 (parsed by the same grammar); their meaning is rung 5, prose, until the elaborator exists.

Claims: D2.1, D2.2, D2.3, D2.4, D2.5, D2.6, D2.7, D2.8, D2.9, D2.10, D2.11, D2.12, D2.13, D2.14, D2.15, D2.16

draft3 — spec

Source: handoff/SHAR-syntax.org, “Draft 3 — paths and directories”; design doc, Draft 3

Why: every example in the syntax documents, in the name spelling (S32), tabs, =;= comment levels (S33), =#= symbols (S29), =$= unquote (S30), =?= goals and =_= inference holes (S40). Each parses under the grammar; the harness reads them from =gram/snips/=. Draft 4 is the pattern-matching sketch (S35–S41). Rung 2 (parsed by the same grammar); their meaning is rung 5, prose, until the elaborator exists.

Claims: D3.1, D3.2, D3.3, D3.4, D3.5

match — spec

Source: handoff/SHAR-syntax.org, “Draft 4 — pattern matching”; design doc, Draft 4

Why: every example in the syntax documents, in the name spelling (S32), tabs, =;= comment levels (S33), =#= symbols (S29), =$= unquote (S30), =?= goals and =_= inference holes (S40). Each parses under the grammar; the harness reads them from =gram/snips/=. Draft 4 is the pattern-matching sketch (S35–S41). Rung 2 (parsed by the same grammar); their meaning is rung 5, prose, until the elaborator exists.

Claims: D4.1, D4.2, D4.3, D4.4, D4.5, D4.6, D4.7, D4.8, D4.9, D4.10, D4.11, D4.12

server — spec

Source: handoff/SHAR-syntax.org, “The counting file server”

Why: every example in the syntax documents, in the name spelling (S32), tabs, =;= comment levels (S33), =#= symbols (S29), =$= unquote (S30), =?= goals and =_= inference holes (S40). Each parses under the grammar; the harness reads them from =gram/snips/=. Draft 4 is the pattern-matching sketch (S35–S41). Rung 2 (parsed by the same grammar); their meaning is rung 5, prose, until the elaborator exists.

sketch — spec

Source: handoff/SHAR-syntax.org, “The planted-tree sketches”

Why: every example in the syntax documents, in the name spelling (S32), tabs, =;= comment levels (S33), =#= symbols (S29), =$= unquote (S30), =?= goals and =_= inference holes (S40). Each parses under the grammar; the harness reads them from =gram/snips/=. Draft 4 is the pattern-matching sketch (S35–S41). Rung 2 (parsed by the same grammar); their meaning is rung 5, prose, until the elaborator exists.